

Complete Fraud Detection Report

Real-Time Banking Transaction Fraud Detection and Decision Support
System

Main author:	AN Duong Binh
Member:	Tuyen Le Quang
Member:	NHI Nguyen Le Hong
Instructor:	Phd. NGU Huynh Cong Viet
Repository:	Final_Project_DMM501_Group1

Submission

focus

Problem definition, architecture, implementation evidence, testing, Responsible AI, and
deployment readiness

Abstract

This report documents a real-time banking transaction fraud detection and decision-support system that combines machine learning scoring, threshold-based policy handling, analyst workflow management, operational monitoring, and Docker-based deployment. The deployed artifact is a logistic regression pipeline trained on the Kaggle credit card fraud dataset, with uncalibrated risk scoring and capacity-driven top-K thresholds aligned to analyst review capacity. Suspicious transactions generate alerts and cases, which are then managed through a case lifecycle and timeline model. The repository also includes a browser-based analyst dashboard, Prometheus and Grafana observability assets, MLflow runtime tracking, SQL-backed persistence support, and CI workflows with an 80% coverage gate. In the current verification run, the test suite completed with 50 passing tests and 80% measured source coverage. This report aligns the final LaTeX version with repository evidence, grading-rubric expectations, deployment screenshots, benchmark artifacts, and honest limitations, so that evaluators can assess both technical implementation quality and end-to-end system maturity.

Contents

1	Executive Summary	1
2	Introduction	1
3	Problem Definition	1
4	Business Context	2
5	Project Scope	2
6	Stakeholders and Actors	3
7	Functional Requirements	4
8	Non-Functional Requirements	5
9	Success Metrics and Target Values	7
10	System Overview	8
11	System Architecture	9
12	Dataset Description	11
12.1	Dataset figures	12
13	Data Challenges	14
14	Machine Learning Pipeline	14
14.1	Experiment tracking and model versioning	14
14.2	Baseline and candidate evidence	15
15	Model Evaluation	17
15.1	Model comparison	17
15.2	Model evaluation figures	18
16	Risk Scoring and Decision Logic	21
17	Fraud Detection Workflow	22
18	Backend System Design	23
18.1	Core request flow: POST /predict	23
18.2	API contract evidence	23
18.3	API endpoint table	23
19	Frontend System	24
19.1	Frontend figures	26
20	Monitoring and Observability	27
20.1	Monitoring figures	28

21 Deployment Architecture	29
21.1 Docker Compose topology (evidence)	30
21.2 Local deployment steps	31
21.3 Containerization and orchestration review	31
21.4 Deployment configuration (selected environment variables)	32
21.5 CI/CD automation (GitHub Actions)	32
21.6 System components table	33
22 Repository and Component Mapping	33
22.1 Repository map	34
22.2 System component map	35
23 Testing and Validation	35
24 Responsible AI	37
25 Results and Discussion	38
26 Documentation and Submission Readiness	38
27 Individual Contribution Evidence	38
28 Rubric Alignment Matrix	39
29 Limitations	40
30 Future Work	41
31 Conclusion	41
32 Documentation Cleanup Summary	41
Implementation Status Matrix	42
A Presentation and Demo Readiness	43
B Submission Hardening Checklist	44

1 Executive Summary

This project implements a fraud detection system that combines machine learning inference with operational decision support for transaction handling. The implemented system accepts a transaction feature vector through a FastAPI service, computes an uncalibrated **risk_score**, maps that score to a **risk_tier** and **decision_recommendation**, and creates an **alert** and **case** for suspicious transactions. The broader system also includes an analyst-facing frontend, case lifecycle management, Prometheus metrics, Grafana dashboards, MLflow runtime tracking, automated tests, and Docker-based deployment components.

The key capability is not only fraud scoring, but the conversion of model output into actionable workflow states. In the implemented design, the machine learning layer acts as a ranking mechanism, while operational services determine whether a transaction should be allowed, sent for manual review, challenged through step-up authentication, held, or blocked. This architecture supports analyst triage, queue management, and investigation tracking rather than treating model output as a final adjudication.

The business objective is to reduce fraud loss, control review workload, and limit unnecessary customer friction. It does so by separating low-risk transactions from review-queue transactions and high-risk transactions, while preserving traceability through **alert**, **case**, and timeline records. The implementation is strongest as a local, end-to-end fraud decision-support demonstration and MLOps reference system, rather than as a production-grade banking control stack.

2 Introduction

Fraud detection remains a central operational problem in banking because digital transactions are high volume, time sensitive, and exposed to multiple attack patterns, including unauthorized card use, account takeover, mule activity, and anomalous transfer behavior. Financial institutions must detect suspicious activity quickly enough to contain loss, while also avoiding unnecessary interruption to legitimate customers.

Traditional rule-based fraud systems remain useful, but they have well-known limitations. Static rules can be brittle, generate excessive false positives, and require substantial manual maintenance as fraud patterns evolve. Rules are also difficult to optimize when risk depends on interactions among many variables rather than a small number of predefined conditions.

Machine learning is therefore useful because it can rank transactions by relative suspiciousness from historical patterns. However, in banking operations, model output alone is insufficient. A practical fraud detection system must integrate scoring with policy decisions, analyst queues, case handling, monitoring, and auditability. This repository reflects that broader framing by combining a supervised fraud model with decision logic, **alert** generation, **case** workflow, and operational interfaces.

3 Problem Definition

The core analytical problem is transaction fraud detection on a highly imbalanced dataset. The repository uses the Kaggle credit card fraud dataset, where fraudulent transactions represent a very small minority of all observations. In such settings, standard accuracy is not meaningful because a naive model can predict the majority class and still appear strong.

The operational problem is more complex than binary classification. A false negative can produce direct financial loss, reputational damage, and delayed investigation. A false positive

can create customer friction, reduce transaction completion, and overload analysts with low-value investigations. For this reason, the implemented system converts model output into `risk_tier` values and downstream `decision_recommendation` values rather than returning only a class label.

The project therefore addresses a decision-support problem: rank transactions by risk, separate suspicious from non-suspicious activity, allocate review capacity using explicit thresholds, and support investigation through `alert` and `case` management.

4 Business Context

Banking fraud controls operate under competing constraints. First, fraud loss matters because delayed intervention can allow rapid cash-out or repeated unauthorized transactions. Second, customer friction matters because unnecessary holds, blocks, or manual reviews can cause legitimate payment failure and dissatisfaction. Third, analyst workload matters because review teams have limited capacity and cannot investigate every transaction manually.

The implemented threshold policy in this repository reflects those constraints. Thresholds are selected using a top-K review-rate policy rather than a single generic cutoff. This means the system is designed to approximate operational review capacity by flagging only the highest-scoring fraction of transactions for review and a smaller fraction for high-risk handling.

The project also acknowledges operational realities beyond scoring. A suspicious transaction must enter an `alert` queue, become a `case`, move through status transitions, and eventually reach a resolution such as `CONFIRMED_FRAUD` or `FALSE_POSITIVE`. Monitoring must then track queue size, status transitions, and false-positive outcomes.

5 Project Scope

The project scope is intentionally broader than a model benchmark and narrower than a production banking platform.

Included in scope

- fraud risk scoring from transaction feature vectors
- explicit `risk_tier` and `decision_recommendation` mapping
- alert and case creation for suspicious transactions
- case lifecycle management and investigation timeline exposure
- frontend support for analyst workflow
- operational monitoring instrumentation
- local deployment assets using Docker Compose
- model training, evaluation, artifact generation, and metadata export

Out of scope

- integration with live core banking transaction streams
- enterprise identity management and secret vault integration
- production-grade model governance, approval workflows, and retraining automation
- direct use of rich real-bank telemetry such as device graphs, merchant history, account network analysis, or customer behavior baselines
- full regulatory compliance packaging for a real bank deployment

Demo-level or prototype-grade areas

- in-memory persistence fallback when no SQL repository is configured
- heuristic reason codes
- locally configured analyst tokens and admin credentials
- static-file-served frontend interaction model

Future work areas

- stronger authentication and secrets management
- richer transaction features and model calibration
- drift detection and closed-loop learning from case outcomes
- broader deployment verification and production runbooks

Assumptions

- transaction requests provide either an ordered feature vector or a feature map aligned to the model metadata
- the deployed artifact metadata remains the source of truth for thresholds and model identity
- analysts interpret outputs as decision support, not as final proof of fraud

6 Stakeholders and Actors

The system involves the following stakeholders and runtime actors.

- Customer: the bank customer initiating a legitimate or fraudulent transaction, indirectly affected by false positives, holds, blocks, and step-up authentication.
- Banking Channel: the upstream transaction source such as mobile app, internet banking, API, ATM, branch, or card-not-present channel, providing the context the system evaluates.
- Fraud API: the backend application that validates requests, executes scoring, applies decision policy, creates workflow records, and exposes operational endpoints.

- Fraud Scoring Engine: the model-loading and inference layer that produces `risk_score` through artifact loading and scoring services.
- Decision Policy Engine: converts `risk_score` into `risk_tier` and `decision_recommendation`, making operational handling explicit rather than implicit.
- Fraud Analyst: reviews `alert` and `case` records, updates case status, adds notes, and resolves investigations using the frontend and case APIs.
- Monitoring / Operations: observes system health, request volume, queue size, prediction distribution, alert creation, and case outcomes through Prometheus and Grafana.
- ML / Model Ops: manages training workflows, artifacts, threshold metadata, and runtime model selection, using artifact outputs and MLflow-related components.

7 Functional Requirements

The following functional requirements are stated in SPS style and aligned to the current repository implementation.

- FR-1: The system shall accept transaction features and return `risk_score`, `risk_tier`, `action`, and `decision_recommendation`.
- FR-2: The system shall document score semantics as uncalibrated risk ranking, not calibrated probability.
- FR-3: The system shall map scores to explicit decision tiers using threshold metadata.
- FR-4: The system shall apply policy-level actions: `LOW` -> `ALLOW`, `REVIEW` -> `STEP_UP_AUTH` or `MANUAL_REVIEW`, `HIGH` -> `HOLD` or `BLOCK`.
- FR-5: The system shall ingest the Kaggle credit card fraud dataset and validate schema before training.
- FR-6: The system shall split data into train, validation, and test partitions using stratification.
- FR-7: The system shall train baseline and improved candidate models and compare them.
- FR-8: The system shall tune decision thresholds using top-K review-rate policy.
- FR-9: The system shall generate reproducible model artifacts and model metadata.
- FR-10: The system shall support `/predict` using either `features` or `features_by_name`.
- FR-11: The system shall expose `/health` with model and threshold metadata.
- FR-12: The system shall expose `/metrics` in Prometheus format.
- FR-13: The system shall expose `/features/schema` for feature contract discovery.
- FR-14: The system shall expose `/stream/pull` for scored event simulation.
- FR-15: The system shall return deterministic HTTP 422 validation errors for malformed inputs.

- FR-16: The system shall return HTTP 503 when no model artifact is loaded.
- FR-17: The system shall expose alert and case workflow endpoints for investigation lifecycle handling.
- FR-18: The system shall enforce endpoint access with configurable authentication and role checks.
- FR-19: The system shall expose audit event listing for governance visibility.
- FR-20: The system shall enforce mutually exclusive input mode (`features XOR features_by_name`).
- FR-21: The system shall return decision explanation fields including recommendation and reason summary.
- FR-22: The system shall support case timeline retrieval for investigations.
- FR-23: The system shall expose internal labeled sample endpoints behind token validation.
- FR-24: The system shall emit Prometheus metrics for requests, latency, and prediction outcomes.
- FR-25: The system shall support Prometheus scraping and Grafana dashboard visualization.
- FR-26: The system shall evaluate configured Prometheus alert rules.
- FR-27: The system shall track queue and case workflow metrics (for example review queue size and case status).
- FR-28: The system shall expose runtime tracking status through health metadata.
- FR-29: The system shall support local deployment using Docker Compose.

Prioritization summary

Priority	Requirement groups
Must have	Accurate fraud scoring, threshold-based decision policy, <code>/predict</code> and <code>/health</code> , alert and case workflow, auditability, monitoring metrics, Docker Compose deployment, automated tests, and documented API contracts.
Should have	SQL-backed persistence, analyst dashboard, stream simulation, MLflow runtime tracking, Prometheus alert rules, and role-based endpoint protection.
Could have	Richer explainability, calibration, queue-aging analytics, drift monitoring, retraining orchestration, and stronger secrets management.
Will not have in this project phase	Core-banking integration, enterprise IAM, fully automated model governance approval, and production regulatory packaging.

Table 1: Requirement prioritization used to keep the implementation aligned with project scope and grading expectations.

8 Non-Functional Requirements

The non-functional requirements are defined as follows.

Performance

- single-request scoring should remain low latency for local/demo operations
- monitoring instrumentation should not materially block request handling
- model startup and readiness should complete within practical local runtime targets

Reliability

- invalid inputs shall produce deterministic validation errors
- case state transitions shall preserve coherent workflow state
- the API shall expose health information and model-loading status

Scalability

- the repository shall support an upgrade path from in-memory demo storage to SQL-backed persistence
- deployment topology shall separate API, persistence, and monitoring services
- services should be container-friendly and horizontally extensible

Observability

- request counts, latency, prediction tiers, case metrics, and review queue size shall be measurable
- monitoring outputs shall support both technical and operational visibility

Maintainability

- the backend shall remain modular across API, services, repositories, and monitoring layers
- explicit schema models shall define public API contracts
- the architecture should preserve service separation for scoring, policy, and workflow modules

Portability

- local development should be supported on Windows and Linux environments
- deployment should be reproducible through Docker Compose
- environment-variable based configuration shall control runtime behavior

Security

- the system shall support token authentication, role-based access control, and rate limiting
- internal labeled-data access shall remain protected
- the current implementation is not a full enterprise security platform and requires further hardening for production use

Usability

- analyst-facing views shall expose queue context, decision support fields, and lifecycle actions with minimal navigation
- analyst workflows should keep key actions (status update, resolve) low-friction

Testability

- the system shall include unit, integration, data, and model tests
- frontend-to-API smoke validation shall be available through dedicated scripts
- deterministic behavior should be preserved where seeded workflows are used

9 Success Metrics and Target Values

To satisfy the grading rubric, success metrics are defined at business, system, and model levels. The targets below are grounded in the current repository configuration, stored artifacts, and monitoring rules rather than being generic textbook goals.

Layer	Metric	Target	Current repository evidence
Business operations	Analyst review rate	At most the top 1% of scored traffic enters REVIEW	Implemented as <code>review_top_rate=0.01</code> in model-selection and deployment metadata.
Business operations	High-risk auto-intervention rate	At most the top 0.2% of scored traffic enters HIGH	Implemented as <code>high_top_rate=0.002</code> in the threshold policy.
Business operations	False-positive surge visibility	Detect spikes before analyst workload becomes unstable	Prometheus alert fires if <code>increase(false_positive_total[30m]) > 25</code> .
System reliability	API error rate	Keep 5xx below 5% over 10 minutes	Prometheus warning rule at <code>> 0.05</code> of request volume.
System performance	API latency	Keep p95 latency below 0.5 seconds	Prometheus warning rule at <code>histogram_quantile(0.95, ...) > 0.5</code> .
System operations	Review queue backlog	Keep active queue below 200 cases	Prometheus warning rule at <code>review_queue_size > 200</code> .
Engineering quality	Automated test coverage	Maintain at least 80% source coverage	Current local verification run achieved exactly 80% with the same threshold enforced in CI.
Model quality	Validation PR-AUC	Stay above 0.62 on validation for model promotion	Selected logistic regression achieved 0.6301 validation PR-AUC.
Model quality	Test recall at review tier	Capture at least 85% of fraud cases sent to analyst review	Test recall at <code>threshold_review</code> is 0.8514.
Model quality	Test precision at high tier	Maintain high-confidence intervention precision above 0.80	Test precision at <code>threshold_high</code> is 0.8429.

Table 2: Project success metrics with explicit targets and current evidence.

10 System Overview

The implemented system follows the pipeline below:

Transaction → Validation → Feature processing → Risk scoring → Decision policy → Alert generation → Case creation → Investigation → Resolution

1. Transaction intake through `features` or `features_by_name`, with optional `transaction_id`, `timestamp`, `amount`, `channel`, and `metadata`.
2. Validation of schema consistency, numeric finiteness, and feature length against model metadata.
3. Feature processing for inference.
4. Risk scoring through the deployed model to obtain an uncalibrated `risk_score`.
5. Decision policy mapping to `risk_tier` and `decision_recommendation`.

6. Reason-code generation through a heuristic decision-support layer.
7. **Alert** and **case** creation for **REVIEW** and **HIGH** outcomes.
8. Analyst investigation through queue, case detail, status changes, and timeline.
9. Resolution through final case outcomes.

This flow is operational rather than purely predictive: the model ranks the transaction, the policy engine assigns handling, workflow services create the case object, and monitoring captures both system and operational health.

11 System Architecture

The architecture integrates data artifacts, an ML pipeline, a decision-support backend, an analyst frontend, monitoring assets, and a deployment stack.

Overall architecture narrative

- Dataset and artifacts: training data is loaded from the credit-card fraud dataset; the pipeline exports trained models, metadata, figures, and reports under **artifacts/**.
- ML pipeline: trains baseline and improved candidate models, selects the deployed model based on validation evidence, and exports threshold metadata and score semantics for runtime use.
- Backend API: validates requests, performs scoring, applies decision logic, creates workflow objects, and exposes analyst and monitoring endpoints.
- Decision layer: maps ranking outputs into operational handling tiers and recommendations.
- Reason code engine: generates heuristic operational explanations and improves analyst interpretation speed without claiming causal explanation.
- Alert and case services: create alerts and cases for flagged transactions and support case lifecycle transitions and timeline access.
- Frontend: provides a live dashboard and review-oriented interaction surface for analysts.
- Monitoring: captures technical and operational metrics and supports queue monitoring, traffic visibility, and case outcome tracking.
- Deployment stack: packages API, frontend, PostgreSQL, Prometheus, Grafana, and MLflow in a local Docker Compose topology.

Rendered architecture diagram

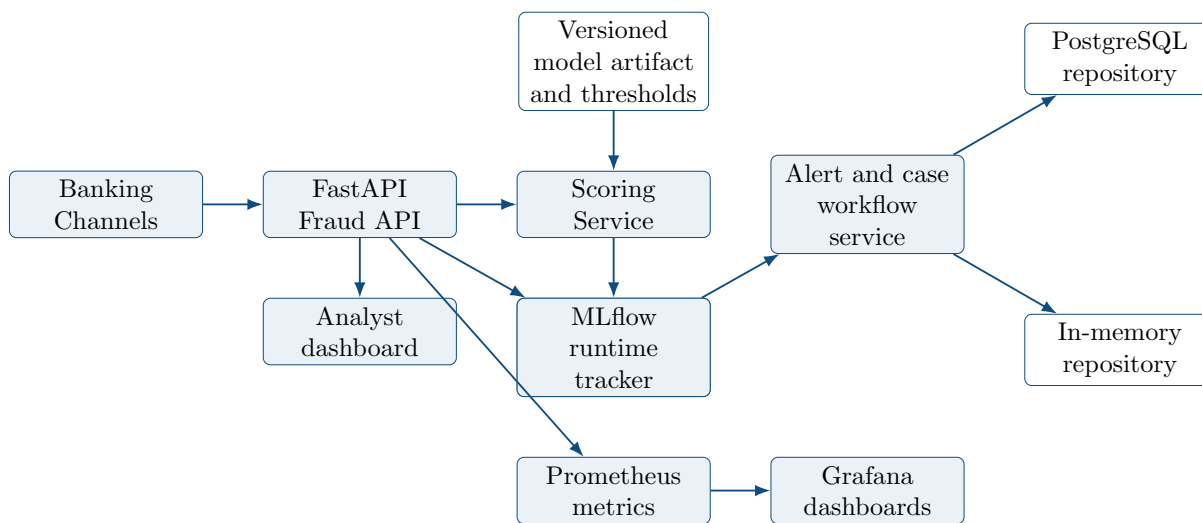


Figure 1: Rendered runtime architecture showing the implemented scoring, workflow, persistence, frontend, and monitoring components.

Transaction data flow and edge-case handling

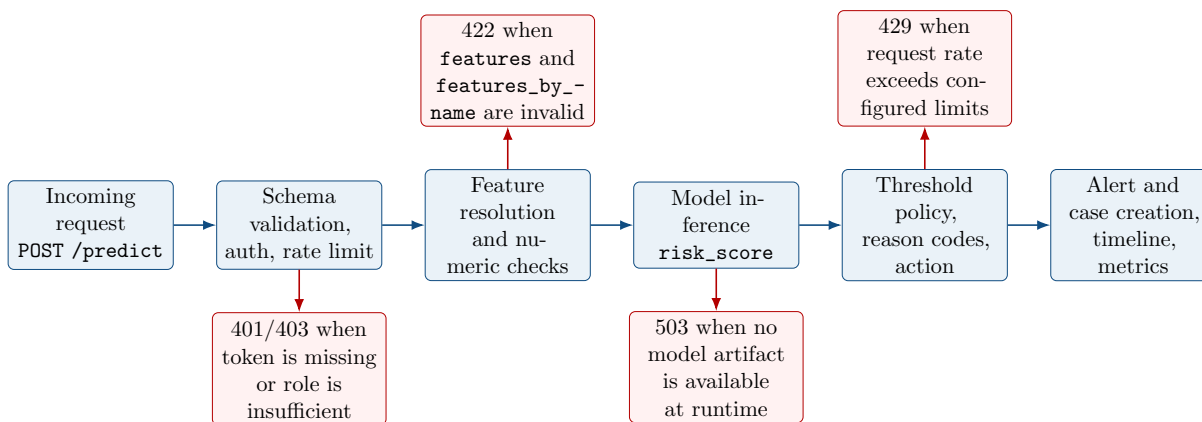


Figure 2: Transaction-serving data flow with explicit handling for malformed input, unauthorized access, missing models, and rate limiting.

Technology decisions and trade-offs

Decision	Why it was chosen	Trade-off and implication
Logistic regression as deployed model	Achieved the best validation PR-AUC (0.6301) and higher review-tier recall than the LightGBM candidate under the same capacity-driven policy.	Less expressive than boosted trees, but simpler to inspect, easier to version, and strong enough on this benchmark.
Top-K threshold policy	Review and high-risk tiers must match analyst capacity, not an arbitrary fixed probability cutoff.	Thresholds are operationally meaningful, but the score must be treated as ranking rather than calibrated probability.
FastAPI for serving	Provides typed request/response schemas, automatic OpenAPI docs, and clean endpoint organization for grading and demo use.	Slightly more framework structure than a minimal Flask service, but much better documentation and validation support.
Repository abstraction with memory and SQL modes	Supports a lightweight demo path while preserving an upgrade path to durable PostgreSQL storage.	Dual persistence modes increase implementation complexity, but make the repo more realistic and easier to demo locally.
Prometheus + Grafana + MLflow stack	Gives both software-operational visibility and ML/runtime tracking using open-source tooling already common in MLOps practice.	More services to run during local demos, but much stronger observability evidence for the final project.

Table 3: Key technical decisions with the business or engineering trade-off behind each choice.

12 Dataset Description

The repository uses the Kaggle credit card fraud dataset stored locally at `data/archive/creditcard.csv` and `data/raw/creditcard.csv`. According to the generated repository artifacts, the dataset contains 284,807 rows and 31 columns, consisting of `Time`, anonymized variables `V1` through `V28`, `Amount`, and the target variable `Class`.

Fraud is extremely rare in the data. The generated class distribution report shows 492 fraud records and 284,315 non-fraud records, giving a fraud ratio of 0.001727485630620034, or approximately 0.17%.

The feature set has important limitations for banking interpretation. Most predictors are PCA-transformed variables rather than business-readable fields such as merchant category, destination account, device reputation, customer tenure, or beneficiary risk.

12.1 Dataset figures

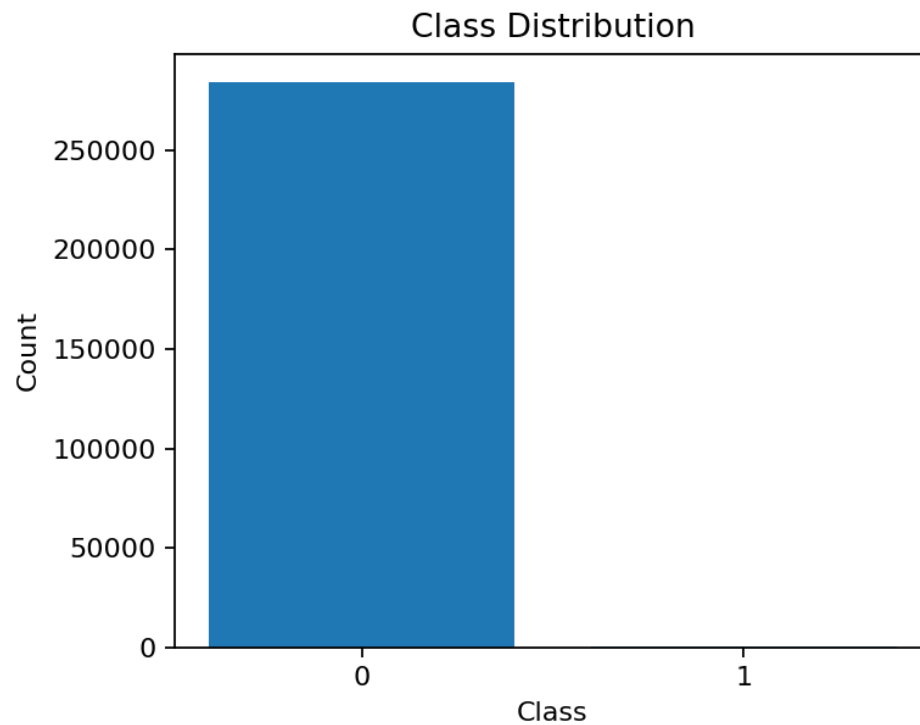


Figure 3: Class imbalance in the benchmark dataset (fraud vs non-fraud).

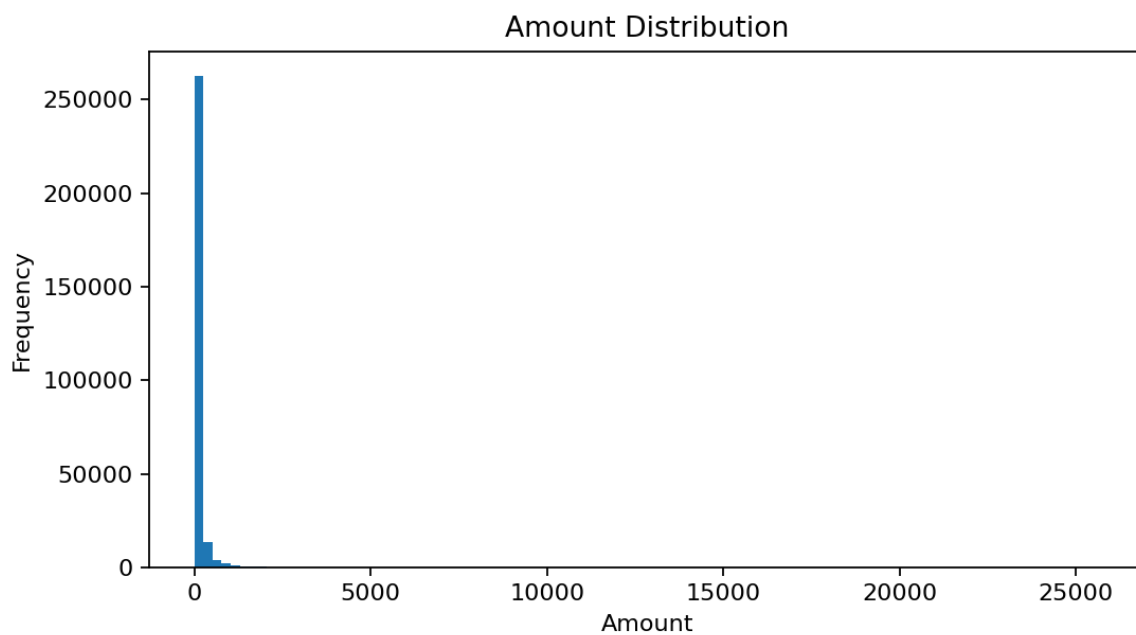


Figure 4: Transaction amount distribution.

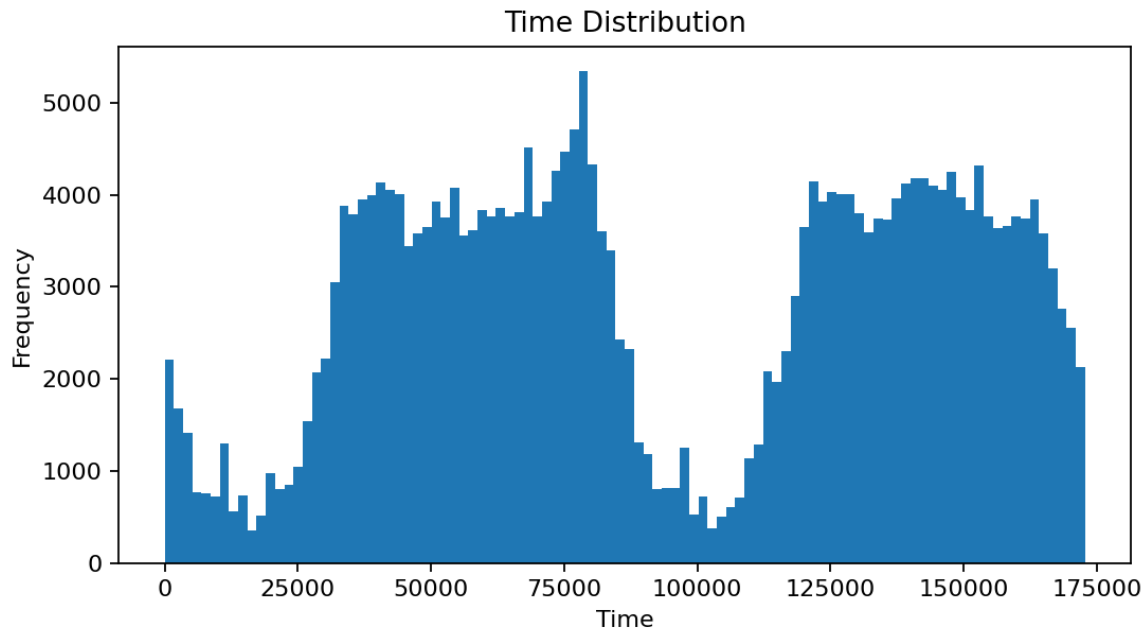


Figure 5: Transaction time distribution.

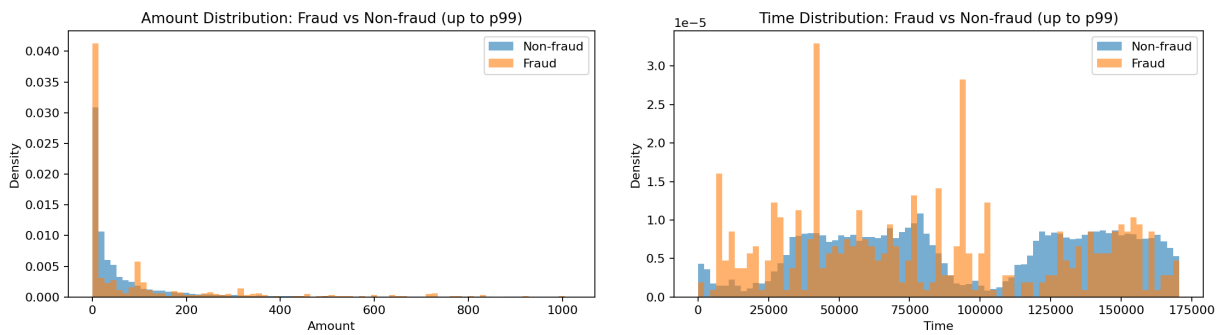


Figure 6: Fraud vs non-fraud comparison for amount (left) and time (right).

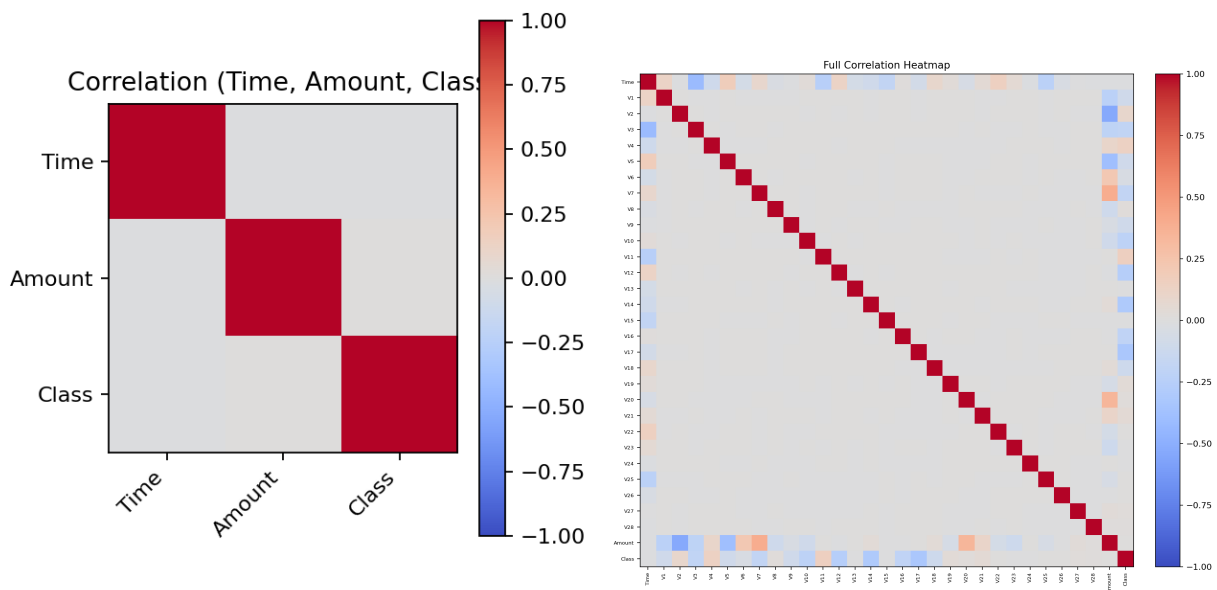


Figure 7: Correlation overview (left) and full matrix view (right).

13 Data Challenges

The first data challenge is extreme class imbalance. Because fraud is rare, a classifier can produce deceptively strong aggregate metrics while still missing many fraudulent transactions or generating too many false alerts. This is why the repository emphasizes PR-AUC, threshold tuning, and operating-point metrics instead of generic accuracy.

The second challenge is interpretability. The PCA-style V1–V28 variables are mathematically useful but operationally opaque. The implemented reason codes therefore rely partly on policy thresholds and optional metadata heuristics rather than on inherently interpretable core features.

The third challenge is simulation realism. Although the project is framed as a banking fraud detection system, the underlying dataset does not contain the full operational context of a bank.

14 Machine Learning Pipeline

The repository contains an explicit training and model-selection workflow in `src/pipelines/run_model_workflow.py` and `src/pipelines/train_pipeline.py`. The workflow reads the credit card dataset, performs exploratory checks and schema validation, splits data into train, validation, and test subsets, and benchmarks multiple models.

The implemented workflow includes preprocessing, model training, evaluation, threshold tuning, and artifact saving. The saved deployment artifact identifies the selected model as a logistic regression pipeline, versioned in the metadata as `v0002_20260424T083412Z`. The threshold policy is explicitly documented in `artifacts/models/model_info.json` as a capacity-driven top-K approach with `review_top_rate=0.01` and `high_top_rate=0.002`.

14.1 Experiment tracking and model versioning

Experiment tracking is implemented at two levels. First, the offline workflow produces benchmark tables, plots, model-selection summaries, and versioned artifacts under `artifacts/`. Second, the

deployed API can emit runtime telemetry into MLflow for traffic and decision monitoring. This gives the project a traceable path from training evidence to serving-time observability.

Tracking element	Repository evidence and interpretation
Model selection basis	<code>artifacts/reports/model_selection_summary.json</code> records that promotion is based on validation PR-AUC, with review-tier recall used as the tie-breaker under the top-K review policy.
Selected artifact	<code>artifacts/models/final_model.joblib</code> and versioned copy <code>artifacts/models/versions/v0002_20260424T083412Z/final_model.joblib</code> .
Model identity	<code>run_id=run_0002_20260424T083412Z</code> , <code>model_version=v0002_20260424T083412Z</code> , and tracking files for latest run and run history.
Threshold metadata	<code>artifacts/models/model_info.json</code> stores <code>threshold_review</code> , <code>threshold_high</code> , score percentiles, and test metrics used by the serving layer.
Hyperparameter evidence	<code>artifacts/benchmarks/improved_hyperparameter_tuning.csv</code> and the saved LightGBM best parameters document the challenger search space.
Runtime observability	Deployment config enables MLflow runtime tracking with experiment <code>fraud-runtime-traffic</code> and run name <code>api-online-traffic</code> .

Table 4: Experiment tracking and model-governance evidence present in the repository.

14.2 Baseline and candidate evidence

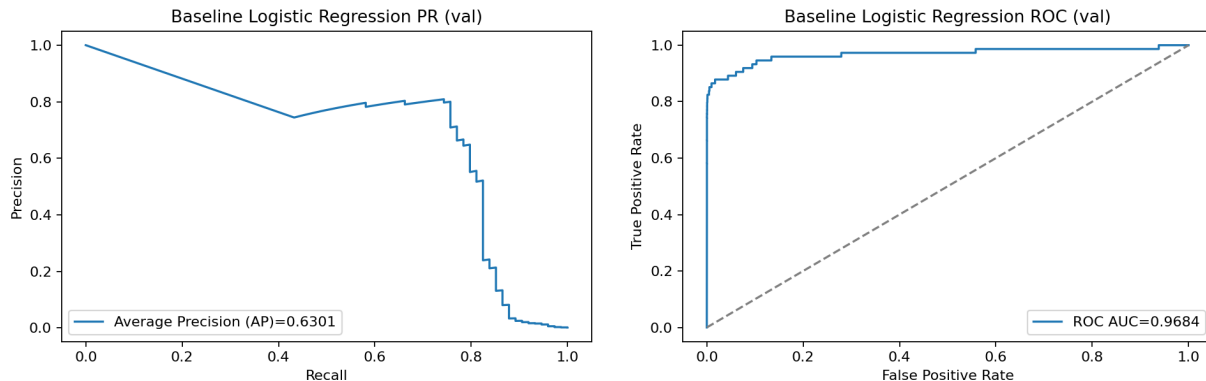
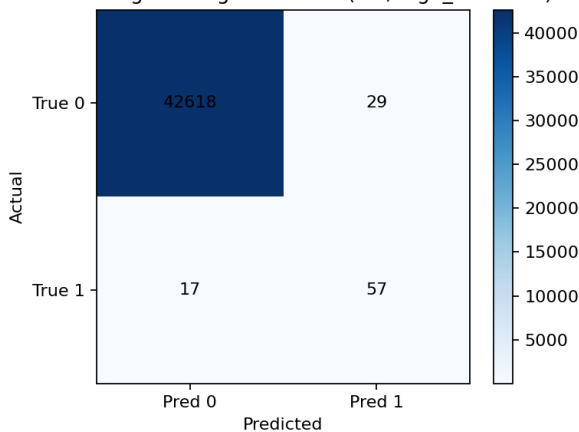
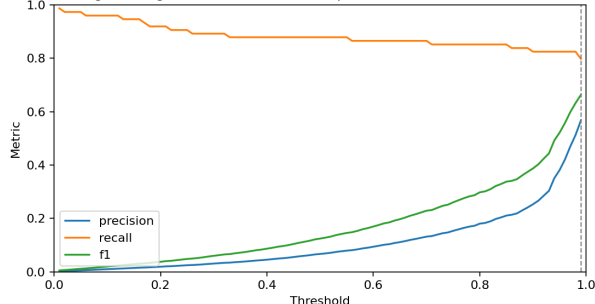
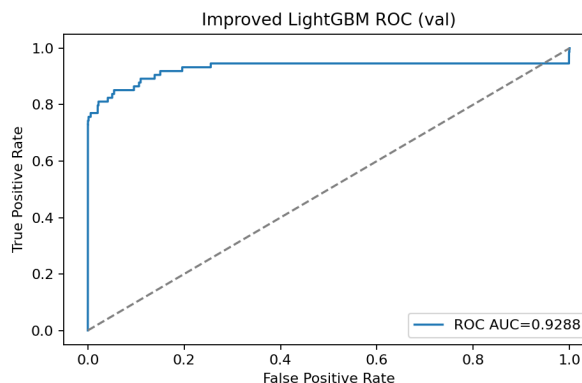
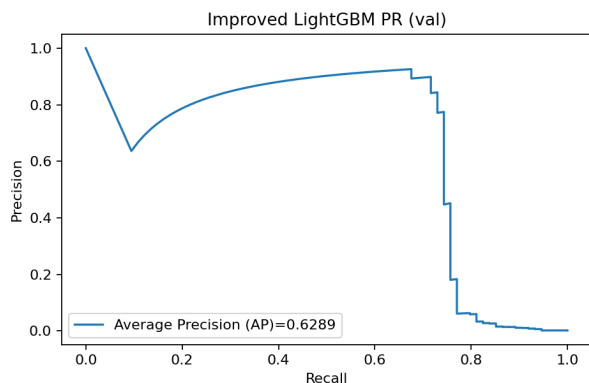


Figure 8: Baseline logistic regression PR curve (left) and ROC curve (right).

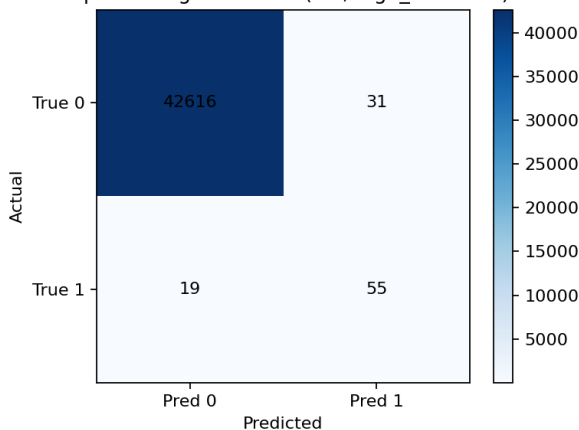
Baseline Logistic Regression CM (val, high_thr=1.00)



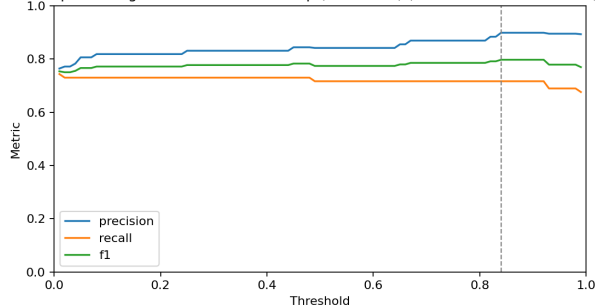
Baseline Logistic Regression Threshold Sweep (validation) (best thr=0.99, f1=0.64)

**Figure 9:** Baseline confusion matrix (left) and threshold sweep (right).**Figure 10:** Improved LightGBM candidate PR curve (left) and ROC curve (right).

Improved LightGBM CM (val, high_thr=0.00)



Improved LightGBM Threshold Sweep (validation) (best thr=0.84, f1=0.797)

**Figure 11:** Improved candidate confusion matrix (left) and threshold sweep (right).

15 Model Evaluation

For fraud detection, PR-AUC is more informative than ROC-AUC because the positive class is extremely rare. ROC-AUC can remain high even when a model produces many false positives at the operating point that matters to analysts. PR-AUC focuses more directly on the precision-recall trade-off under class imbalance.

15.1 Model comparison

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression (Review)	0.7391	0.1470	0.8510	0.2510	0.9684	0.6300
LightGBM (Review)	0.0000004	0.1330	0.7700	0.2270	0.9288	0.6290

Table 5: Validation operating-point comparison from repository benchmark artifacts.

The logistic regression row is the currently deployed artifact; the LightGBM row remains the strongest challenger retained for benchmark comparison.

The selection rationale is important for defendability. Logistic regression was promoted because it slightly outperformed the LightGBM candidate on validation PR-AUC (0.6301 vs 0.6289) and achieved materially stronger review-tier recall (0.8514 vs 0.7703) at the review operating point. That decision is aligned with the project goal of maximizing suspicious-transaction capture under a fixed analyst-capacity budget, even if the more complex model could still be useful as a future challenger.

The repository also stores the final deployed evaluation metrics at the operating thresholds used for decision support. At the review threshold, precision is 0.1462 and recall is 0.8514. At the high threshold, precision is 0.8429 and recall is 0.7973. These figures show the trade-off between broader queue capture at the review threshold and tighter containment at the high threshold.

15.2 Model evaluation figures

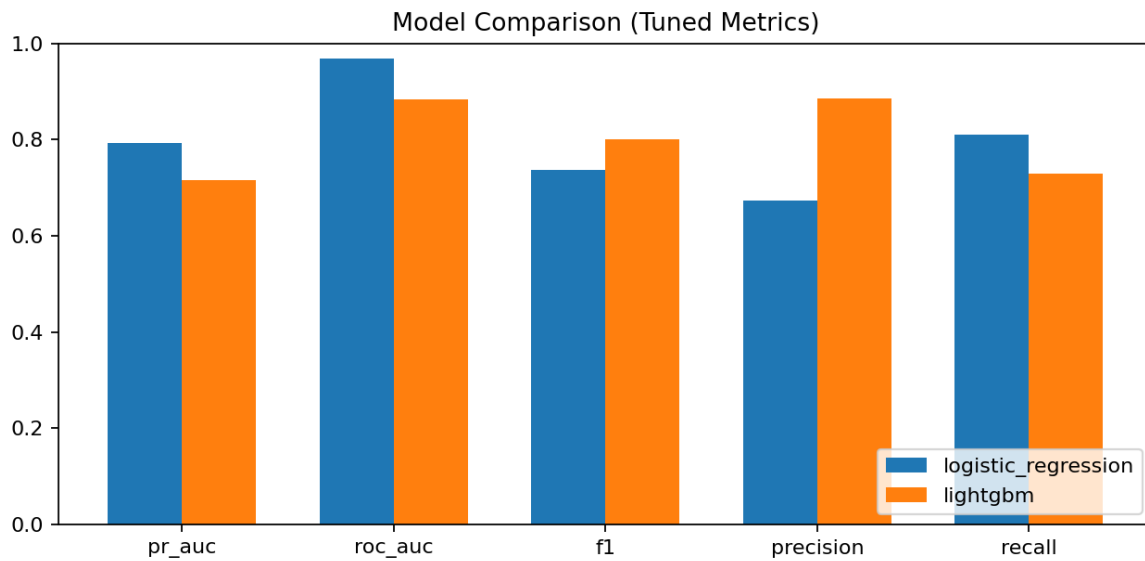


Figure 12: Model benchmark comparison summary from saved artifacts.

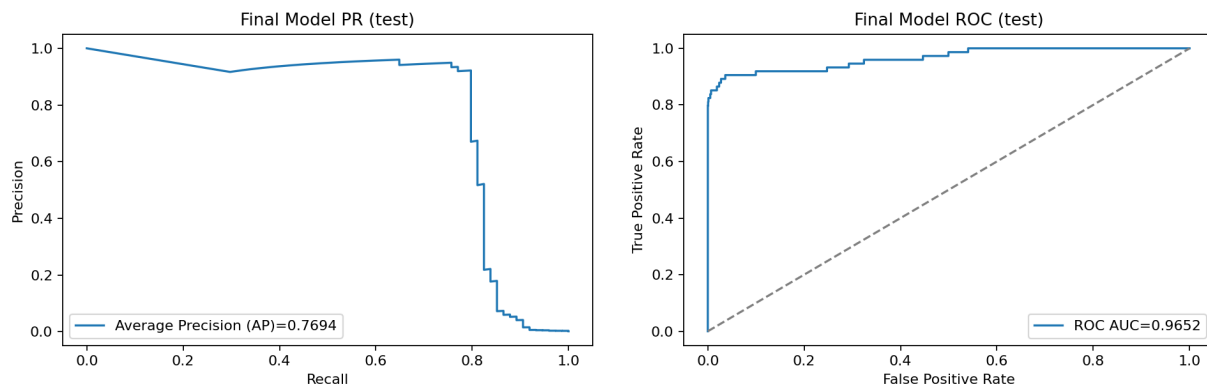


Figure 13: Final deployed model PR curve (left) and ROC curve (right).

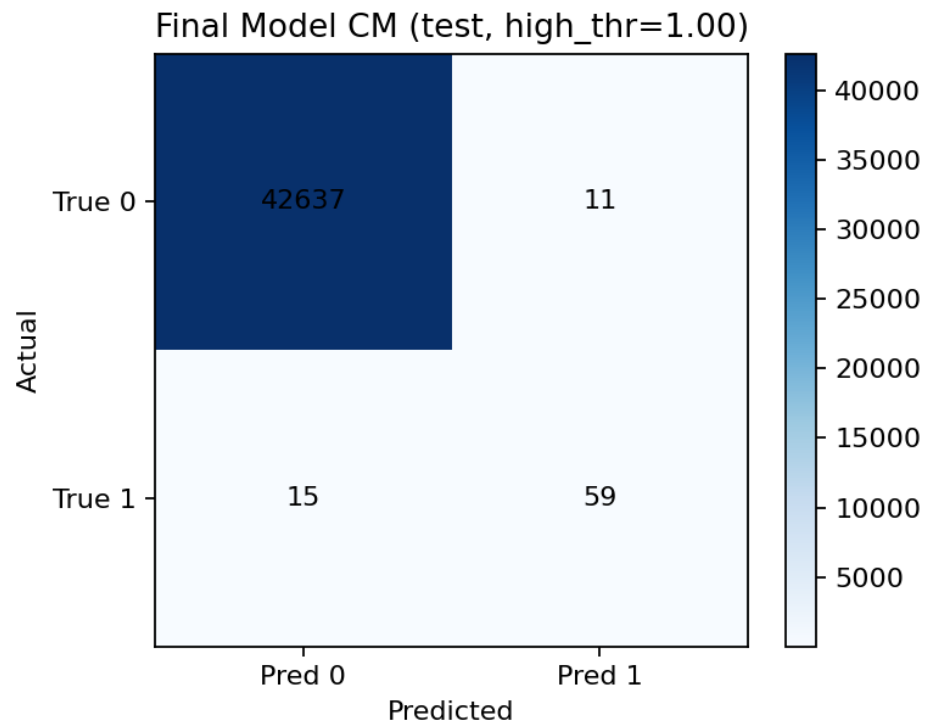


Figure 14: Final deployed model confusion matrix at the selected operating point.

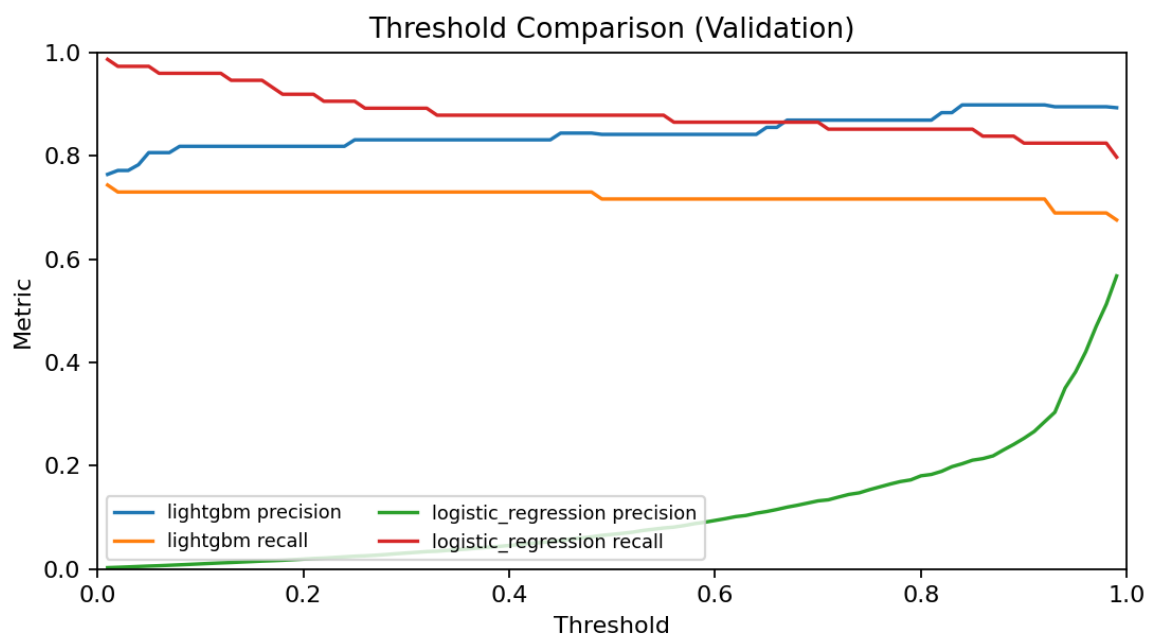


Figure 15: Threshold comparison evidence supporting review/high policy tiers.

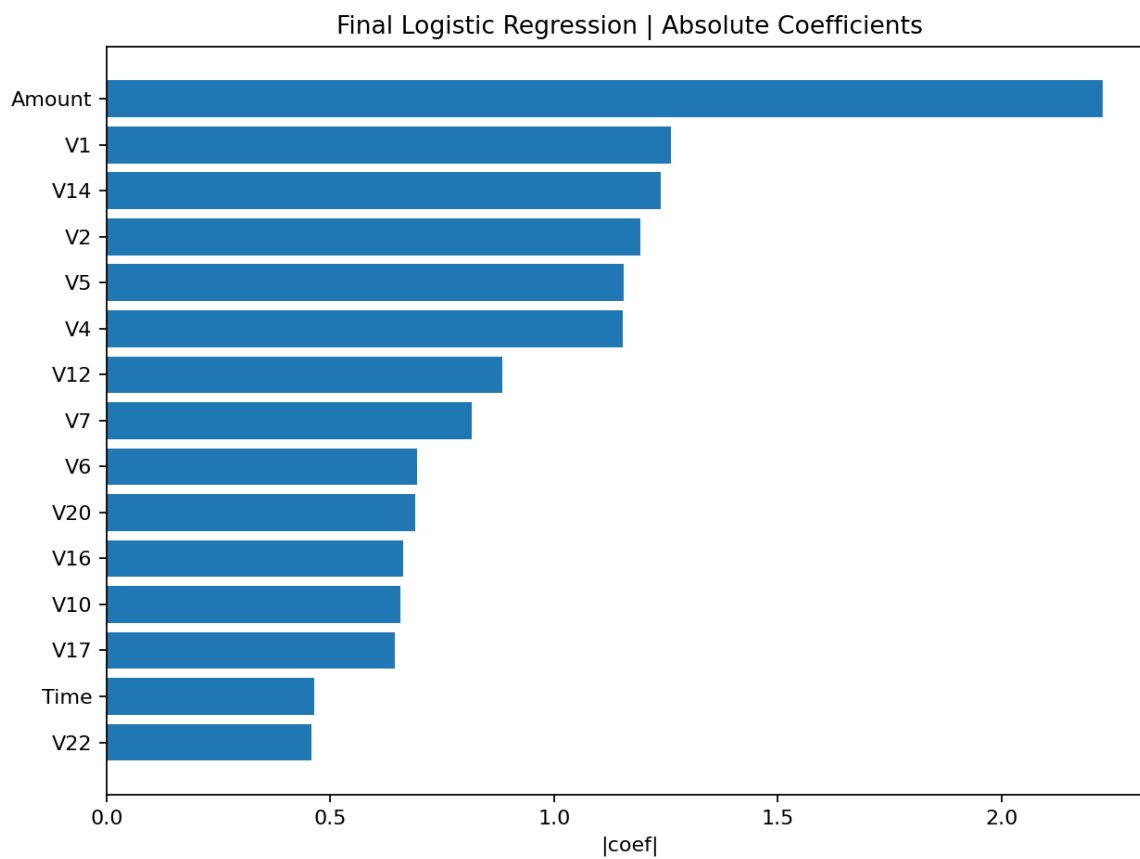


Figure 16: Feature importance evidence from saved artifacts (model-dependent).

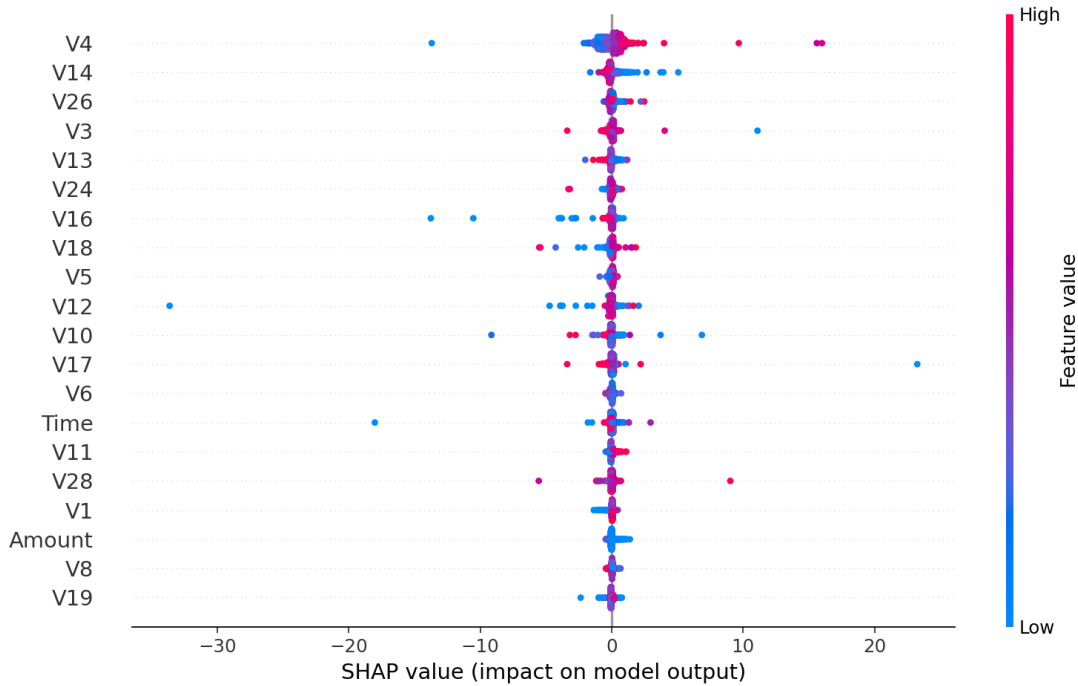


Figure 17: SHAP summary plot from saved artifacts (interpretation aid; not causal proof).

16 Risk Scoring and Decision Logic

The API exposes `risk_score` in the range $[0, 1]$, but this score is explicitly described in the repository as uncalibrated. It should therefore be interpreted as a ranking signal rather than a calibrated fraud probability.

The deployed thresholds are:

- review threshold: 0.7391262534904675
- high-risk threshold: 0.9999047447184487

The implemented `risk_tier` logic is:

- LOW if `risk_score < threshold_review`
- REVIEW if `threshold_review <= risk_score < threshold_high`
- HIGH if `risk_score >= threshold_high`

The implemented `decision_recommendation` logic is:

- ALLOW for LOW
- STEP_UP_AUTH for digital-channel REVIEW
- MANUAL_REVIEW for non-digital-channel REVIEW
- HOLD for default HIGH
- BLOCK for HIGH when amount is at least 1500.0

Score condition	risk_tier	decision_recommendation
<code>< threshold_review</code>	LOW	ALLOW
<code>>= threshold_review and < threshold_high</code>	REVIEW	STEP_UP_AUTH or MANUAL_REVIEW
<code>>= threshold_high</code>	HIGH	HOLD or BLOCK

Table 6: Score-to-decision mapping.

17 Fraud Detection Workflow

The implemented fraud detection workflow is as follows:

1. A transaction arrives through POST `/predict`.
2. The backend validates the request and extracts the feature vector.
3. The scoring service computes the uncalibrated `risk_score`.
4. The decision service assigns a `risk_tier` and `decision_recommendation`.
5. The reason-code service generates decision-support cues.
6. If the result is `REVIEW` or `HIGH`, the case service creates an `alert` and linked `case`.
7. The transaction enters the fraud alert queue exposed by GET `/alerts` and GET `/cases`.
8. An analyst can move the `case` into `IN_REVIEW`, investigate it, update status, and resolve the outcome.
9. Timeline events record the sequence from scoring to closure.

The implemented `case` lifecycle statuses are `NEW`, `QUEUED`, `IN_REVIEW`, `ESCALATED`, `CONFIRMED_FRAUD`, `FALSE_POSITIVE`, `BLOCKED`, `RELEASED`, and `RESOLVED`. Timeline events include `TRANSACTION_RECEIVED`, `RISK_SCORED`, `FLAGGED`, `ALERT_CREATED`, `CASE_ASSIGNED`, `INVESTIGATION_STARTED`, resolution events, and `CASE_CLOSED`.

Stage	Implemented behavior	Output
Transaction intake	API accepts features and optional transaction context	Validated request
Risk scoring	Model computes uncalibrated <code>risk_score</code>	Score in <code>[0, 1]</code>
Decision policy	Score mapped to <code>risk_tier</code> and <code>decision_recommendation</code>	Operational guidance action
Alert creation	<code>REVIEW</code> and <code>HIGH</code> create <code>alert</code> and <code>case</code>	Queue item
Investigation	Analyst updates status and reviews reason codes and timeline	Ongoing <code>case</code>
Resolution	Analyst marks confirmed fraud, false positive, blocked, released, or resolved	Closed outcome

Table 7: Fraud workflow summary.

18 Backend System Design

The backend is implemented as a FastAPI application. It loads the trained model at startup, instantiates a stream simulator when a model is available, and constructs a case repository through an environment-driven repository factory.

The key backend services are the scoring service, decision service, reason code service, and case service. The core scoring endpoint is `POST /predict`. Supporting endpoints include health, metrics, stream simulation, feature schema access, dataset sampling, alert retrieval, case retrieval, status updates, case resolution, and timeline retrieval.

18.1 Core request flow: `POST /predict`

At a high level, the backend converts a prediction request into operational outputs through the following deterministic sequence.

```
request -> validate schema + auth + rate limit
        -> resolve features (features OR features_by_name)
        -> validate feature vector length and numeric finiteness
        -> score_transaction(model, features) -> risk_score
        -> decide_risk_action(risk_score, thresholds) -> risk_tier, action, recommendation
        -> generate_reason_codes(...) -> reason_codes + summary
        -> if risk_tier in {REVIEW, HIGH}: create alert + case + timeline events
        -> record Prometheus metrics + audit event
        -> return PredictResponse(...)
```

Key implementation references:

- request handling: `src/api/main.py`
- scoring: `src/services/scoring_service.py`
- decision logic: `src/services/decision_service.py`
- workflow orchestration: `src/services/case_service.py`
- persistence: `src/repositories/`

18.2 API contract evidence

18.3 API endpoint table

Endpoint	Method	Purpose	Status
<code>/health</code>	GET	API and model metadata, thresholds, queue size, persistence mode	Implemented
<code>/features/schema</code>	GET	Return expected feature contract	Implemented
<code>/features/random</code>	GET	Return demo feature vectors	Implemented
<code>/metrics</code>	GET	Prometheus metrics export	Implemented
<code>/predict</code>	POST	Score transaction and optionally create alert and case	Implemented
<code>/stream/pull</code>	GET	Pull pre-scored demo events	Implemented
<code>/alerts</code>	GET	List fraud alerts	Implemented

Endpoint	Method	Purpose	Status
/alerts/<alert_id>	GET	Retrieve alert detail	Implemented
/alerts/<alert_id>/status	POST	Update linked case status via alert	Implemented
/cases	GET	List cases	Implemented
/cases/<case_id>	GET	Retrieve case detail	Implemented
/cases/<case_id>/status	POST	Update case lifecycle status	Implemented
/cases/<case_id>/resolve	POST	Resolve case outcome	Implemented
/cases/<case_id>/timeline	GET	Retrieve investigation timeline	Implemented
/audit/events	GET	Retrieve audit events	Implemented, ancillary
/dataset/samples	GET	Fetch unlabeled sample rows	Implemented
/internal/dataset/samples	GET	Fetch labeled sample rows with token protection	Implemented, internal

For storage, the system supports two repository modes. The in-memory repository is fully implemented and used as the default demo mode. The SQL repository is implemented in code, covered by integration testing, and wired into Docker Compose through PostgreSQL. SQL-backed persistence is configurable and operational when enabled.

19 Frontend System

The frontend is an analyst-facing dashboard implemented in static HTML, CSS, and JavaScript. It connects to the backend API, displays model and threshold metadata, supports live streaming views, and provides a review workflow for suspicious transactions.

The implemented frontend includes a dashboard view for live stream control and summary KPIs, a real-time transaction feed, a fraud alert queue, a review view with case filtering and paging, a case detail panel, and case status actions such as `IN_REVIEW`, `CONFIRMED_FRAUD`, `FALSE_POSITIVE`, and resolution actions.

The frontend also displays a clear score semantics cue stating that the score is uncalibrated rather than a true fraud probability. The UI is therefore more than a visualization layer; it acts as the client for analyst operations against the backend's `alert` and `case` APIs.

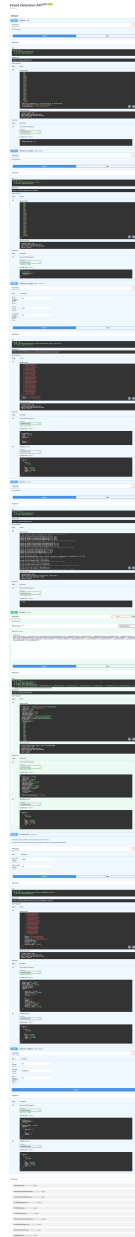


Figure 18: Swagger/OpenAPI documentation for the FastAPI backend (local deployment evidence screenshot).

19.1 Frontend figures

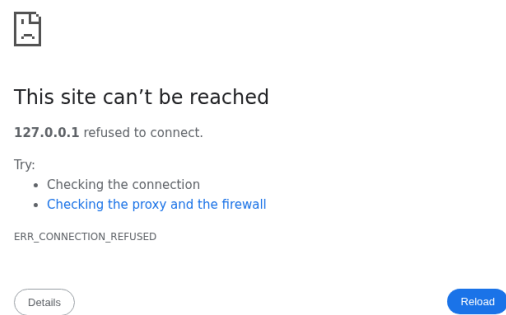


Figure 19: Analyst dashboard view (local deployment evidence screenshot).

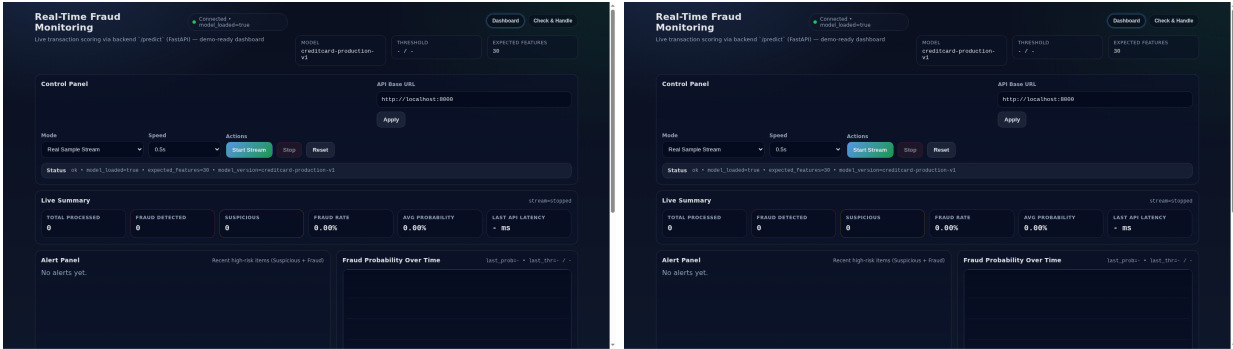


Figure 20: Frontend live-mode dashboard (left) and streaming view (right).

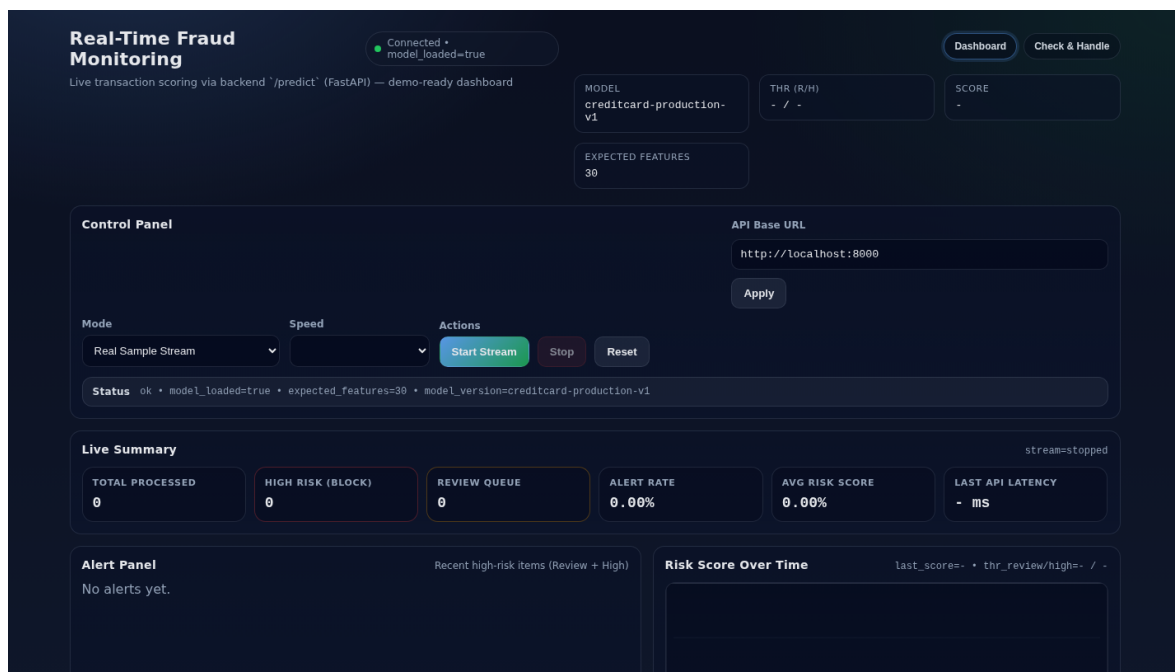


Figure 21: Frontend streaming interaction view.

20 Monitoring and Observability

The backend exposes Prometheus metrics and includes Grafana dashboard configuration plus MLflow runtime tracking. Monitoring is critical in fraud systems because operational failure can arise from more than model degradation. High latency, queue buildup, false-positive spikes, or unusually low scoring traffic can each undermine fraud operations even when the model artifact itself is unchanged.

The implemented Prometheus metrics include API request volume and status counts, API latency histograms, fraud prediction counts by tier, `risk_tier` distribution, `decision_recommendation` distribution, `alert` and `case` creation counts, `case` status transition counts, `review_queue_size`, confirmed fraud counts, false-positive counts, and risk score aggregates.

The implemented Prometheus alert rules include high 5xx error rate, high p95 API latency, unusually low prediction traffic, unusually high prediction traffic, review queue backlog, and false-positive spike.

MLflow runtime tracking is also implemented. The tracker records runtime traffic metrics such as total requests, predict calls, stream pulls, scored events, average `risk_score`, tier totals, HTTP status families, and decision totals.

Alerting thresholds and operational meaning

Alert rule	Threshold	Operational meaning
FraudApiHigh5xxErrorRate	> 5% of requests over 10 minutes	Detects unstable API behavior before prediction service quality collapses.
FraudApiHighP95Latency	p95 > 0.5 seconds for 10 minutes	Protects real-time decision support from degraded response times.
FraudApiPredictionRateTooLow	< 0.01 predictions/sec for 15 minutes	Surfaces stalled traffic, broken upstream feeds, or idle demo environments.
FraudApiPredictionRateTooHigh	> 50 predictions/sec for 10 minutes	Surfaces abnormal traffic spikes or abuse scenarios.
FraudReviewQueueBacklogHigh	review_queue_size > 200 for 15 minutes	Signals that analyst capacity is being exceeded.
FraudFalsePositiveSpike	More than 25 false positives in 30 minutes	Highlights model/policy drift or poor analyst experience.

Table 9: Configured Prometheus alert rules and the operational risk each rule addresses.

20.1 Monitoring figures

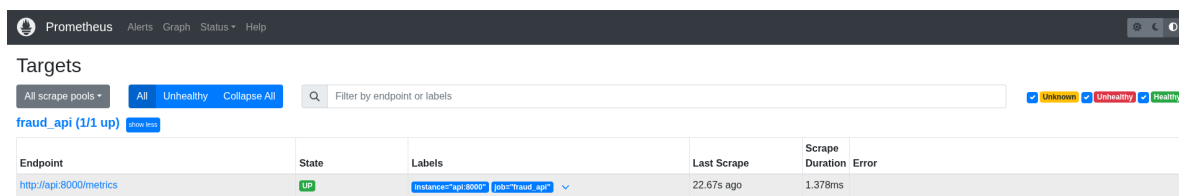


Figure 22: Prometheus scrape targets (deployment evidence screenshot).

Rules				
fraud-api-alerts		Interval: 1m 0s	18.828s ago	0.562ms
Rule	State	Error	Last Evaluation	Evaluation Time
alert: <code>FraudApiHigh5xxErrorRate</code> expr: <code>(sum(rate(api_requests_total{http_status=~"5.."}[5m])) / clamp_min(sum(rate(api_requests_total[5m])), 1)) > 0.05</code> for: 10m labels: severity: warning annotations: description: More than 5% of requests are 5xx over the last 10 minutes. summary: High 5xx error rate on Fraud API	OK		18.830s ago	0.323ms
alert: <code>FraudApiHighP95Latency</code> expr: <code>histogram_quantile(0.95, sum by (le) (rate(api_request_latency_seconds_bucket[5m])) > 0.5</code> for: 10m labels: severity: warning annotations: description: p95 request latency is above 0.5 seconds over the last 10 minutes. summary: High p95 latency on Fraud API	OK		18.830s ago	0.118ms
alert: <code>FraudApiPredictionRateTooLow</code> expr: <code>sum(rate(fraud_predictions_total[5m])) < 0.01</code> for: 15m labels: severity: info annotations: description: Prediction rate is below 0.01 req/s (near idle) for 15 minutes. summary: Prediction traffic is very low	OK		18.830s ago	0.046ms
alert: <code>FraudApiPredictionRateTooHigh</code> expr: <code>sum(rate(fraud_predictions_total[5m])) > 50</code> for: 10m labels: severity: warning annotations: description: Prediction rate is above 50 req/s for 10 minutes. summary: Prediction traffic is unusually high	OK		18.830s ago	0.034ms

Figure 23: Prometheus alert rules loaded and evaluated (deployment evidence screenshot).

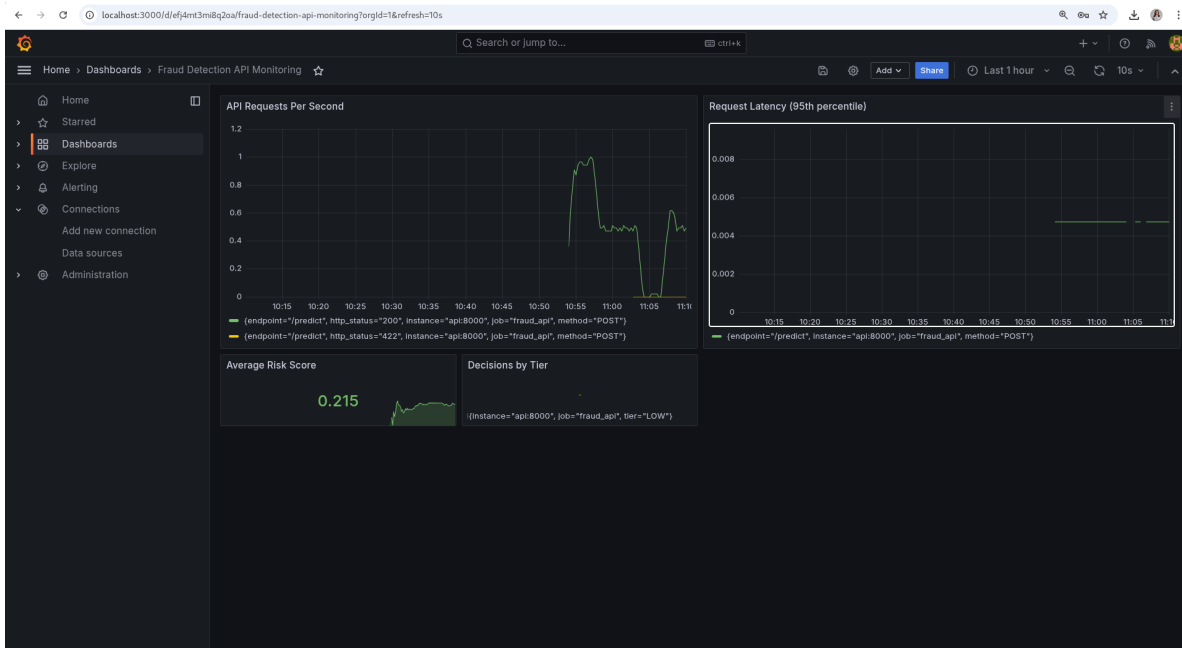


Figure 24: Grafana monitoring dashboard for the fraud API (deployment evidence screenshot).

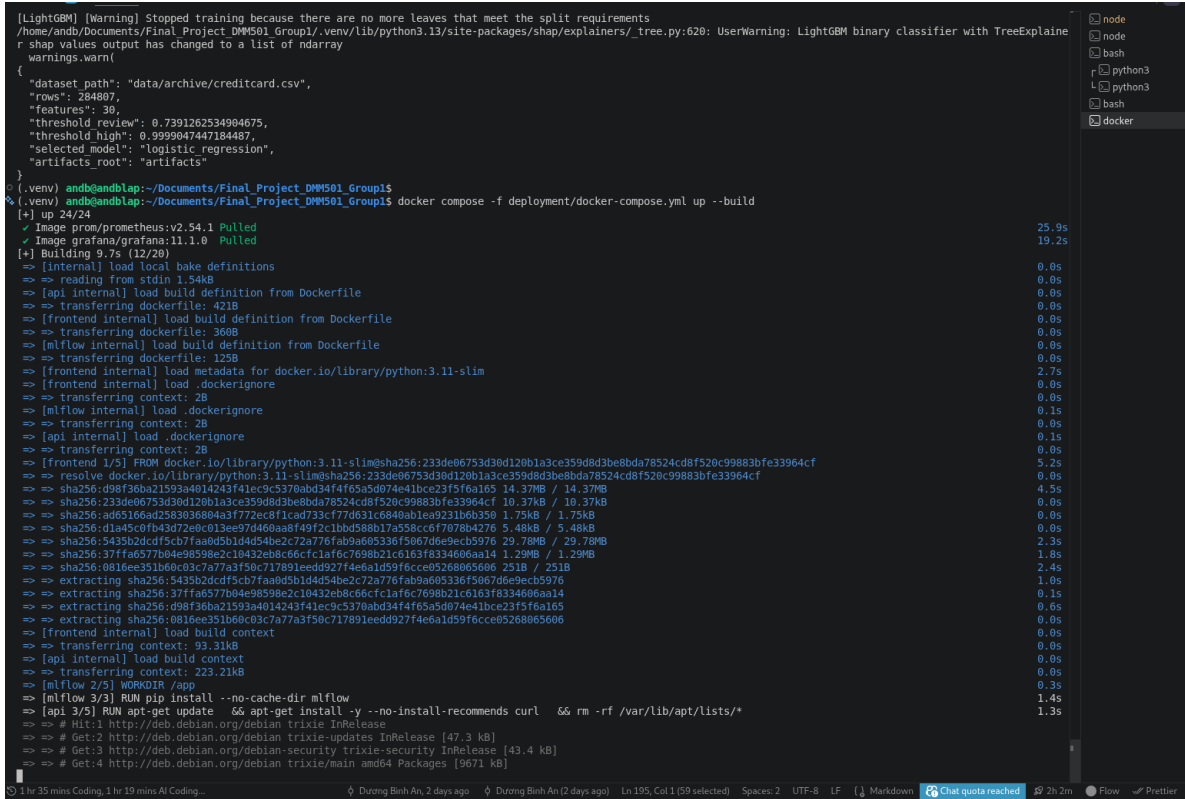
21 Deployment Architecture

The repository includes a Docker Compose deployment that orchestrates the broader system end to end. The defined services are API, frontend, Prometheus, Grafana, MLflow, and PostgreSQL.

The API container loads the saved fraud model and mounts the artifacts volume. The frontend container

serves the analyst dashboard. Prometheus scrapes the API metrics endpoint and loads alert rules. Grafana reads provisioned dashboards. MLflow hosts experiment and runtime tracking. PostgreSQL provides the durable repository option when the API is configured for SQL persistence.

21.1 Docker Compose topology (evidence)



```
[LightGBM] [Warning] Stopped training because there are no more leaves that meet the split requirements
/home/andb/Documents/Final_Project_DMM501_Group1/venv/Lib/python3.13/site-packages/shap/explainers/_tree.py:620: UserWarning: LightGBM binary classifier with TreeExplainer
  shap values output has changed to a list of ndarray
warnings.warn(
{
  "dataset_path": "data/archive/creditcard.csv",
  "rows": 284897,
  "features": 30,
  "threshold_review": 0.7391262534904675,
  "threshold_high": 0.9999047447184487,
  "selected_model": "logistic_regression",
  "artifacts_root": "artifacts"
}
(.venv) andb@andbap:~/Documents/Final_Project_DMM501_Group1
% (.venv) andb@andbap:~/Documents/Final_Project_DMM501_Group1 docker compose -f deployment/docker-compose.yml up --build
[+] up 24/24
  ✓ Image prom/prometheus:v2.54.1 Pulled      25.9s
  ✓ Image grafana/grafana:11.1.0 Pulled      19.2s
[+] Building 9.7s (12/20)
=> [internal] load local bake definitions      0.0s
=> reading from stdin 1.54kB                  0.0s
=> [api internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 421B              0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 360B              0.0s
=> [mlflow internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 125B              0.0s
=> [frontend internal] load metadata for docker.io/library/python:3.11-slim 2.7s
=> [frontend internal] load .dockerignore      0.0s
=> transferring context: 2B                   0.0s
=> [mlflow internal] load .dockerignore       0.1s
=> transferring context: 2B                   0.0s
=> [api internal] load .dockerignore          0.1s
=> transferring context: 2B                   0.0s
=> [frontend 2/5] FROM docker.io/library/python:3.11-slimsha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 0.0s
=> resolve docker.io/library/python:3.11-slimsha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 0.0s
=> sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 14.37kB / 10.37kB 4.5s
=> sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 10.37kB / 10.37kB 0.0s
=> sha256:ad65166ad2583036804a3f772ec8f1cad733cf77d631c6840ab1ea9231b6b350 1.75kB / 1.75kB 0.0s
=> sha256:d4e51c0f0bbd72eb013ee97d460a8d4492c21b4d580b17a538c6f7070b4226 5.40kB / 5.40kB 0.0s
=> sha256:5435b2dcd5c7f7faad5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.79kB / 29.79kB 0.0s
=> sha256:37ffa6577b04e989598e2c10432eb8c66cf1af6c7698b21c616f8334606aa14 1.29MB / 1.29MB 1.8s
=> sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 2.51B / 2.51B 2.4s
=> extracting sha256:5435b2dcd5c7f7faad5b1d4d54be2c7a776fab9a605336f5067d6e9ecb5976 1.0s
=> extracting sha256:37ffa6577b04e989598e2c10432eb8c66cf1af6c7698b21c616f8334606aa14 0.1s
=> extracting sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 0.6s
=> extracting sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 0.0s
=> [frontend internal] load build context      0.0s
=> transferring context: 93.31kB              0.0s
=> [api internal] load build context          0.0s
=> transferring context: 223.21kB            0.0s
=> [mlflow 2/5] WORKDIR /app                 0.3s
=> [mlflow 3/3] RUN pip install --no-cache-dir mlflow 1.4s
=> [api 5/5] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* 1.3s
=> # Hit:1 http://deb.debian.org/debian trixie InRelease
=> # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> # Get:3 http://deb.debian.org/debian trixie-security InRelease [43.4 kB]
=> # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
```

Figure 25: Docker Compose topology screenshot (deployment evidence).

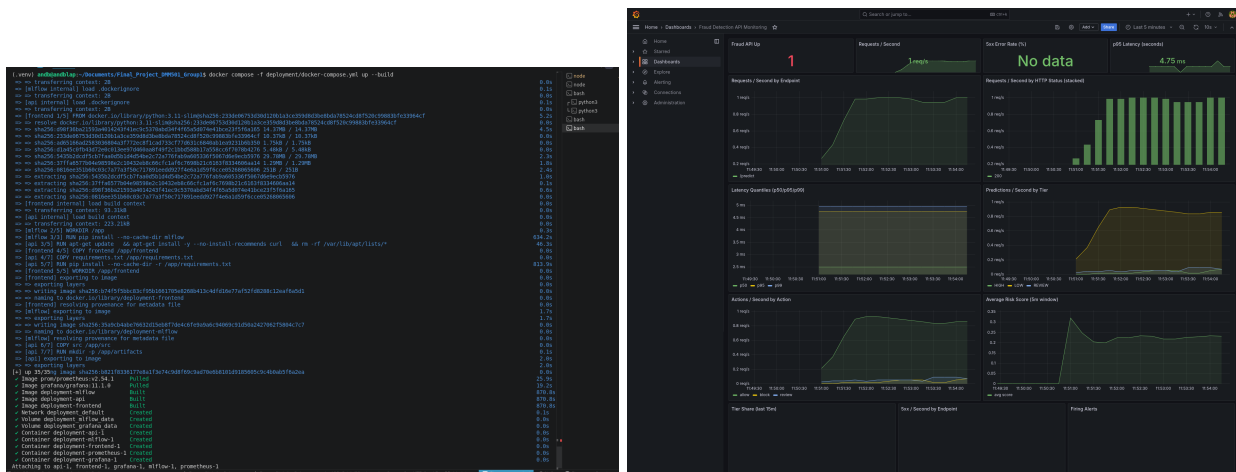


Figure 26: Local container startup evidence (left) and monitoring dashboard evidence (right).

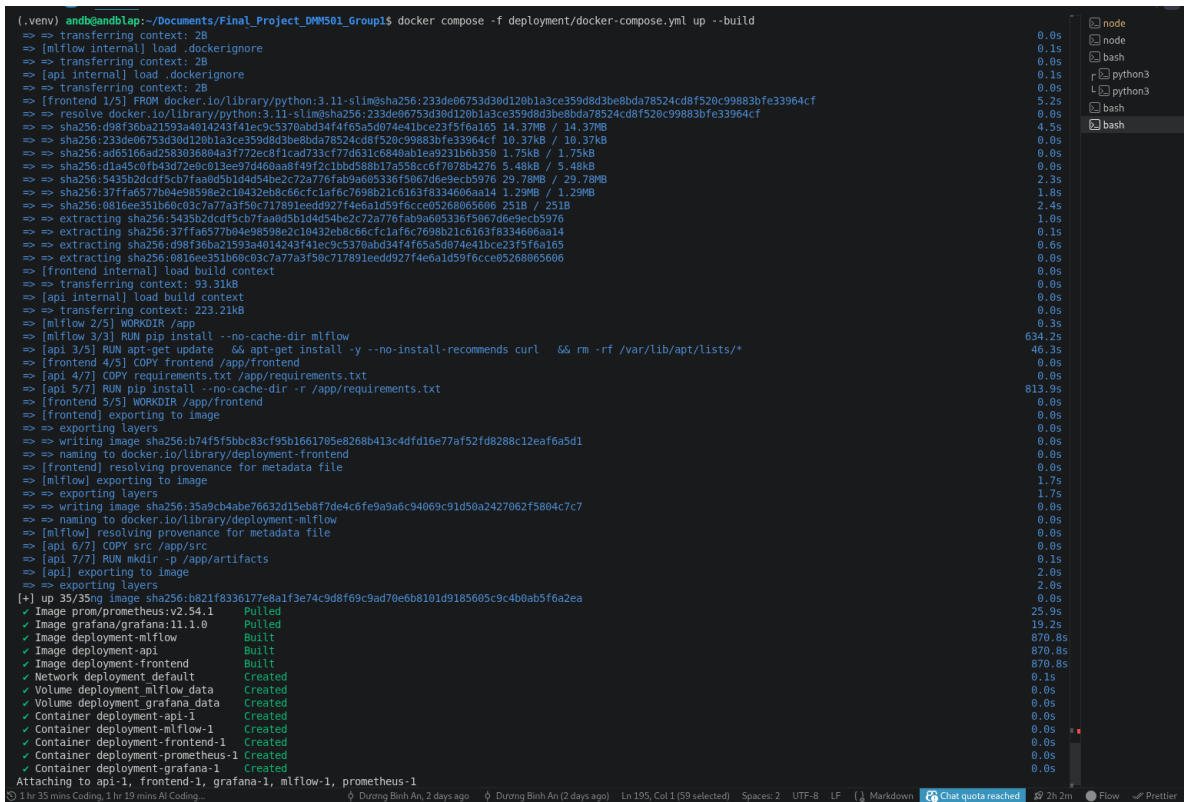
21.2 Local deployment steps

The recommended demo deployment path is Docker Compose:

```
# From repo root:
docker compose -f deployment/docker-compose.yml up --build
```

Key service URLs (default ports):

- API docs: <http://localhost:8000/docs>
- Frontend: <http://localhost:8082/>
- Prometheus: <http://localhost:9090/>
- Grafana: <http://localhost:3000/> (admin password configured by GF_SECURITY_ADMIN_PASSWORD)
- MLflow: <http://localhost:5000/>



```
(.venv) and@andlaptop:~/Documents/Final_Project_DMM501_Group1$ docker compose -f deployment/docker-compose.yml up --build
=> transferring context: 2B
=> [mlflow internal] load .dockerignore
=> transferring context: 2B
=> [api internal] load .dockerignore
=> transferring context: 2B
=> [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
=> resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
=> sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5fe1a65 14.37MB / 14.37MB
=> sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 10.37KB / 10.37KB
=> sha256:ad65166ad2583036804a3f772ec8f1cad733cf77d631c6840ab1ea92316b350 1.75KB / 1.75KB
=> sha256:d1a45cfb4d72ec013ee9f4d60a8f49f2c1bbd588b1f558cc5f7070a4276 5.48KB / 5.48KB
=> sha256:5435b2dcdcf5c77faad5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.78MB / 29.78MB
=> sha256:37ffa6577b04e9898e2c10432eb8c66cfc1af6c7698b21c6163f334606aa14 1.29MB / 1.29MB
=> sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 251B / 251B
=> extracting sha256:5435b2dcdcf5c77faad5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976
=> extracting sha256:37ffa6577b04e9898e2c10432eb8c66cfc1af6c7698b21c6163f334606aa14
=> extracting sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5fe1a65
=> extracting sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606
=> [frontend internal] load build context
=> transferring context: 93.31kB
=> [api internal] load build context
=> transferring context: 223.21kB
=> [mlflow 2/5] WORKDIR /app
=> [mlflow 3/3] RUN pip install --no-cache-dir mlflow
=> [api 3/5] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
=> [frontend 4/5] COPY frontend /app/frontend
=> [api 4/7] COPY requirements.txt /app/requirements.txt
=> [api 5/7] RUN pip install --no-cache-dir -r /app/requirements.txt
=> [frontend 5/5] WORKDIR /app/frontend
=> [frontend] exporting to image
=> exporting layers
=> writing image sha256:b74f5f5b8c83cf95b1661705e8268b413c4df016e77af52fd8288c12eaf6a5d1
=> naming to docker.io/library/deployment-frontend
=> [frontend] resolving provenance for metadata file
=> [mlflow] exporting to image
=> exporting layers
=> writing image sha256:35a9cb4abe76632d15eb8f7de4c6fe9a9a6c94069c91d50a247062f5804c7c
=> naming to docker.io/library/deployment-mlflow
=> [mlflow] resolving provenance for metadata file
=> [api 6/7] COPY src /app/src
=> [api 7/7] RUN mkdir -p /app/artifacts
=> [api] exporting to image
=> exporting layers
=> writing image sha256:b221f8336177e8a1f3e74c9d8f69c9ad70e6b810d9185605c9c4b0ab5f6a2ea
[+] up 35/35s Image sha256:b221f8336177e8a1f3e74c9d8f69c9ad70e6b810d9185605c9c4b0ab5f6a2ea
✔ Image prom/prometheus:v2.54.1 Pulled
✔ Image grafana/grafana:11.1.0 Pulled
✔ Image deployment-mlflow Built
✔ Image deployment-api Built
✔ Image deployment-frontend Built
✔ Network deployment-default Created
✔ Volume deployment-mlflow-data Created
✔ Volume deployment-grafana-data Created
✔ Container deployment-api-1 Created
✔ Container deployment-mlflow-1 Created
✔ Container deployment-frontend-1 Created
✔ Container deployment-prometheus-1 Created
✔ Container deployment-grafana-1 Created
Attaching to api-1, frontend-1, grafana-1, mlflow-1, prometheus-1
1 hr 35 mins Coding, 1 hr 19 mins AI Coding...
Dương Binh An, 2 days ago Dương Binh An (2 days ago) Ln 195, Col 1 (59 selected) Spaces: 2 UTF-8 LF Markdown Chat quota reached 2h 2m Flow Prettier
```

Figure 27: Local docker compose up and running services (deployment evidence).

21.3 Containerization and orchestration review

The deployment implementation is stronger than a single-container demo because it separates runtime concerns into purpose-specific services and wires them together with Compose health checks and dependency conditions. PostgreSQL exposes a readiness probe using `pg_isready`; the API and frontend each have HTTP health checks; and the API waits for PostgreSQL to become healthy before startup. This design supports a much more realistic local demo than a notebook-plus-server setup.

Deployment area	quality	Current repository status
Service decomposition		Implemented for API, frontend, PostgreSQL, Prometheus, Grafana, and MLflow in one Compose topology.
Health checks		Implemented for PostgreSQL, API, and frontend, improving demo stability and restart behavior.
Environment-driven configuration		Implemented for model path, repository mode, auth, rate limits, and runtime tracking.
Image strategy		Functional slim Python images are used for API and frontend serving. They are reproducible, but future hardening can still add non-root execution and multi-stage optimization.
Operational realism		Strong for a course project because monitoring, persistence, dashboards, and experiment tracking run together instead of being described only on paper.

Table 10: Assessment of containerization and orchestration quality in the current repository.

21.4 Deployment configuration (selected environment variables)

The deployment uses environment variables to control model selection, persistence mode, authentication, and monitoring integration. The most important variables are:

Variable	Meaning
MODEL_PATH	Path to the deployed artifact inside the API container (mounted from <code>artifacts/</code>).
MODEL_VERSION	A human-readable identifier returned in <code>/health</code> and responses for traceability.
CASE_REPOSITORY_MODE	<code>memory</code> (demo) or <code>postgres</code> (durable storage).
CASE_DB_URL	SQLAlchemy connection URL for PostgreSQL when <code>postgres</code> is enabled.
CASE_DB_AUTO_MIGRATE	When <code>true</code> , applies DB schema migrations at startup (demo convenience).
API_AUTH_ENABLED	When enabled, requires a token for protected endpoints.
API_TOKENS	Token-to-role mapping used for viewer/analyst/admin authorization.
RATE_LIMIT_ENABLED	Enables request rate limiting middleware.
MLFLOW_RUNTIME_ENABLED	Enables MLflow runtime traffic tracking.
MLFLOW_RUNTIME_TRACKING_URI	MLflow server URI used by the runtime tracker.

Table 11: Selected deployment environment variables.

21.5 CI/CD automation (GitHub Actions)

The repository includes CI workflows for Python tests and container build validation:

- `.github/workflows/ci.yml`: unit tests, integration tests, and a coverage gate.
- `.github/workflows/docker.yml`: Docker image builds for API and frontend, plus `docker compose config` validation.

Workflow job	Trigger and platform	What it validates
<code>unit-tests</code>	Push or pull request on <code>ubuntu-latest</code>	Runs <code>pytest -q tests/unit tests/data</code> to catch fast failures in preprocessing, IDs, data sampling, and core utilities.
<code>integration-and-coverage</code>	Push or pull request on <code>ubuntu-latest</code>	Runs API integration tests and then enforces <code>--cov-fail-under=80</code> across the codebase.
<code>build-and-validate</code>	Push or pull request on <code>ubuntu-latest</code>	Builds API and frontend images and validates Compose configuration syntax for deployment readiness.

Table 12: CI/CD workflow coverage in the repository’s GitHub Actions configuration.

21.6 System components table

Component	Role	Evidence in repository	Status
Logistic regression model	Fraud scoring	<code>artifacts/models/final_model.joblib</code> , <code>model_info.json</code>	Implemented
Decision policy service	Map score to tier and recommendation	<code>src/services/decision_service.py</code>	Implemented
Reason code service	Produce decision-support cues	<code>src/services/reason_code_service.py</code>	Implemented, heuristic
FastAPI backend	Serve scoring and workflow APIs	<code>src/api/main.py</code>	Implemented
In-memory repository	Default alert and case storage	<code>src/repositories/in_memory_case_repository.py</code>	Implemented, demo-level
SQL/Postgres repository	Durable case storage path	<code>src/repositories/sql_case_repository.py</code>	Implemented (configurable)
Frontend dashboard	Analyst queue and case handling UI	<code>frontend/</code>	Implemented
Prometheus metrics	Operational observability	<code>src/monitoring/metrics.py</code>	Implemented
Grafana dashboards	Visualization layer	<code>deployment/grafana/</code>	Implemented
MLflow tracking	Experiment and runtime tracking	<code>deployment/mlflow/</code> , <code>src/monitoring/mlflow_runtime_tracker.py</code>	Implemented

Table 13: System components and implementation status.

22 Repository and Component Mapping

The repository is organized around a source-code core, a static frontend, generated artifacts, deployment assets, tests, and documentation. The tables below summarize the most important areas and the runtime components they support.

22.1 Repository map

Folder / Area	Layer	Responsibility	Key Files / Notes
<code>src/</code>	Core application	Backend logic for scoring, decisions, workflow, monitoring, and security	<code>src/api/main.py</code> , <code>src/services/decision_service.py</code> , <code>src/repositories/factory.py</code>
<code>frontend/</code>	Presentation	Analyst dashboard, API client, and review workflow UI	<code>frontend/index.html</code> , <code>frontend/app.js</code> , <code>frontend/api-client.js</code>
<code>artifacts/</code>	ML and evidence	Models, figures, benchmark tables, reports, deployment evidence, and MLflow data	<code>artifacts/models/</code> , <code>artifacts/reports/</code>
<code>deployment/</code>	Deployment and observability	Dockerfiles, Compose stack, Prometheus, Grafana, and MLflow assets	<code>deployment/docker-compose.yml</code> , <code>deployment/prometheus/</code> , <code>deployment/grafana/</code>
<code>tests/</code>	Validation	Unit, data, integration, and smoke validation	<code>tests/integration/</code> , <code>tests/unit/</code> , <code>tests/verify_system.py</code>
<code>.github/workflows/</code>	CI/CD support	Repository automation definitions	<code>.github/workflows/ci.yml</code> , <code>.github/workflows/docker.yml</code>
<code>docs/</code>	Documentation area	Legacy project documentation location	Legacy content retained for reference; the consolidated narrative lives in this report

Table 14: Repository-to-responsibility mapping.

22.2 System component map

Component	Type	Responsibility	Status / note
Scoring service	Backend service	Produces risk_score from transaction features	Fully implemented
Decision policy engine	Backend service	Maps risk_score to risk_tier and decision_recommendation	Fully implemented
Reason code engine	Backend service	Generates heuristic explanation signals	Partially implemented
Case service	Workflow service	Creates and updates case records and lifecycle state	Fully implemented
Timeline service	Workflow feature	Exposes investigation event history	Fully implemented
Frontend dashboard	UI	Displays stream, cases, and review actions	Fully implemented
Monitoring stack	Operations	Exposes metrics and dashboards	Fully implemented
SQL persistence path	Infrastructure	Supports durable case storage	Fully implemented
In-memory fallback	Demo infrastructure	Supports local/demo case storage	Demo-level

Table 15: Component-level mapping for the runtime system.

23 Testing and Validation

The repository contains unit, data, model, integration, security, frontend/API, and SQL-persistence tests. The inventory covers preprocessing behavior, request ID generation, dataset path handling and sampling logic, model training artifact creation, probability range validation, key API endpoints, alert and case lifecycle flow, token-based authorization, audit events, rate limiting, SQL repository persistence across app restarts, and frontend-to-API interaction.

Current verification result

In the current workspace, the full test command below was executed successfully:

```
./venv/bin/pytest -q --cov=src --cov-config=.coveragerc --cov-report=term-missing
```

That verification run produced **50 passed tests** and **80% total source coverage**. This matters because the repository CI pipeline uses the same coverage threshold as its merge gate.

Test category	Representative files	What it validates
Unit tests	tests/unit/test_loader_and_scoring.py, test_reason_code_service.py	Core scoring behavior, reason-code logic, preprocessing, and ID generation.
Data tests	tests/data/test_samples.py	Dataset path handling, sample retrieval, and data utility behavior.
Model tests	tests/model/test_model_validation_placeholder.py	Training artifact creation, metric sanity, and model probability range checks.
Integration tests	tests/integration/test_api_predict_happy_path.py, test_api_alert_case_workflow.py, test_api_sql_persistence.py	Test-to-end API behavior, security checks, workflow state transitions, and SQL persistence.
Frontend/API smoke tests	tests/test_frontend_api.py	Browser-facing integration assumptions between dashboard behavior and backend endpoints.

Table 16: Test strategy by test type and repository coverage area.

Coverage highlights and residual gaps

Area	Coverage	Interpretation
src/api/schemas.py	100%	Public request/response contracts are fully exercised, which is important for API reliability.
src/services/reason_code_service.py	98%	Decision-support explanation logic is well covered.
src/monitoring/metrics.py	98%	Metric emission paths are directly tested, supporting observability claims.
src/models/loader.py	95%	Artifact loading and metadata handling are strongly validated.
src/sql_case_repository.py	57%	Durable persistence exists and is integration-tested, but branch coverage for all SQL paths can still be expanded.
src/monitoring/mlflow_runtime_tracker.py	38%	Runtime tracking exists and is wired, but deserves deeper direct unit coverage in a future hardening pass.

Table 17: Selected coverage observations from the latest test run.

The latest run also surfaced a small number of non-fatal warnings: sklearn feature-name warnings during some model tests and resource warnings around SQLite connections. These warnings do not invalidate the passing result, but they identify good cleanup opportunities before final submission.

24 Responsible AI

The repository is explicit that the system is a decision-support system and not a fully autonomous fraud adjudication engine. This framing is important because banking fraud operations are high stakes: incorrect blocking can harm legitimate customers, while missed fraud can produce direct financial loss and regulatory exposure. The project therefore treats the model as a ranking component inside a broader human-supervised workflow.

Fairness and representational limits

The available dataset does not include explicit protected attributes such as gender, age band, ethnicity, or disability status. That means the project cannot claim measured demographic fairness, and it would be technically weak to pretend otherwise. The more defensible position is that fairness risk is acknowledged, its absence from the dataset is documented, and the system is designed so that analysts can override model outputs through case review rather than automatically finalizing outcomes.

Explainability methods

Explainability is implemented at two levels:

- model-level interpretation through SHAP summary artifacts and feature-importance exports
- operational explanation through heuristic reason codes, score tiering, and explicit decision recommendations returned by the API

This is useful but intentionally limited. SHAP helps analysts and reviewers see which transformed features contribute most strongly to model behavior, while reason codes help operational triage. Neither should be described as causal proof because the underlying PCA-style features are anonymized and not business-native.

Ethical controls and human oversight

Risk area	Current mitigation	Residual limitation
False positives harming legitimate customers	Review and high-risk tiers are separated; analysts can resolve cases as <code>FALSE_POSITIVE</code> or <code>RELEASED</code> .	Manual review still consumes analyst time and can still create customer friction.
Over-trust in raw model score	Report and frontend explicitly state that <code>risk_score</code> is uncalibrated and should be treated as ranking, not probability.	Users may still misinterpret a value near 1.0 without additional calibration or percentile display.
Opaque model behavior	SHAP summary, feature importance, and reason-code outputs provide partial interpretability.	PCA-based features remain difficult to explain in business language.
Unreviewed automated decisions	Final case outcomes are produced through case workflow, not by the model alone.	High-risk responses such as <code>HOLD</code> or <code>BLOCK</code> are still policy-driven and would need governance review in production.

Table 18: Responsible AI controls, mitigations, and known residual limitations.

25 Results and Discussion

The project demonstrates that a relatively simple model can perform well on the benchmark dataset when paired with explicit threshold policy and workflow design. The saved artifacts show strong ROC-AUC and meaningful PR-AUC, while the review and high-risk thresholds provide different operating points for queueing and containment.

The system's main strength is integration. It aligns model output, backend orchestration, frontend handling, and monitoring into a coherent fraud decision-support process. The inclusion of **alert** generation, **case** creation, status transitions, timeline events, Prometheus metrics, and Docker deployment makes the repository materially stronger than a standalone notebook or scoring endpoint. The saved artifacts also confirm that logistic regression is the selected deployed artifact in the current workflow, not the LightGBM candidate, because it achieved slightly better validation PR-AUC.

26 Documentation and Submission Readiness

The repository already contains the files required by the submission checklist: `README.md`, `ARCHITECTURE.md`, `CONTRIBUTING.md`, `requirements.txt`, container definitions under `deployment/`, and CI workflows under `.github/workflows/`. FastAPI also exposes live OpenAPI documentation at `/docs`, which strengthens the API documentation criterion beyond a static endpoint list.

Submission requirement	Status	Evidence or note
<code>README.md</code>	Present	Includes project overview, setup path, runtime flow, screenshots, and validation snapshot.
<code>ARCHITECTURE.md</code>	Present	Documents actors, runtime flow, lifecycle, monitoring, and deployment topology.
<code>CONTRIBUTING.md</code>	Present	Branching, testing, and CI expectations are documented; team-specific evidence links should be finalized before submission.
<code>requirements.txt</code>	Present	Python dependencies for training, serving, tests, and monitoring are tracked in one file.
Dockerfile and Compose	Present	API, frontend, MLflow, and Compose topology are implemented under <code>deployment/</code> .
<code>.github/workflows/</code>	Present	<code>ci.yml</code> and <code>docker.yml</code> provide automated validation on push and pull request.
API documentation	Present	FastAPI Swagger/OpenAPI docs are available and already evidenced in this report.

Table 19: Submission-readiness checklist aligned to the project brief.

27 Individual Contribution Evidence

The project brief allows individual grades to move based on commit history, role clarity, and live Q&A. The current repository history already provides usable contribution evidence, but it should be presented carefully and honestly.

Visible identity	git author	Commits in current his- tory	Representative work visible in commit subjects
Duong Binh An		23	Integration test fixtures, dataset path handling, model loading tests, MLflow runtime tracking, frontend/API verification, Docker/frontend updates, model workflow, refactors, and final report integration.
Annwyn		20	Documentation expansion, benchmark summaries, system specification, CI coverage updates, deployment scripts, monitoring exporter, Grafana/dashboard work, artifact refreshes, and frontend/API system verification.

Table 20: Contribution evidence visible directly from the current git history.

For course submission, the team should ensure that official member names and GitHub identities are mapped clearly. The title page lists the official roster as AN Duong Binh, Tuyen Le Quang, and NHI Nguyen Le Hong, while the git history currently exposes the author identities **Duong Binh An** and **Annwyn**. If those names correspond to different course members or aliases, the final submission packet should attach PR links or a short identity-mapping note so grading remains fair and defensible.

28 Rubric Alignment Matrix

The table below makes the grading argument explicit. It does not claim perfection; rather, it maps each rubric area to concrete repository evidence that evaluators can verify quickly.

Rubric area	Repository evidence	Why this is defensible in presentation
Problem definition and requirements	Problem framing, business context, prioritized functional and non-functional requirements, and explicit success metrics are documented in the first third of this report.	Demonstrates that the project is solving an operational fraud decision problem, not only a binary-classification exercise.
System design and architecture	Rendered architecture diagrams, transaction data flow with edge cases, and a trade-off table are included in the architecture section.	Gives evaluators both the big-picture topology and the reasoning behind the chosen design.
Implementation: ML pipeline	Multiple candidate models, selection artifacts, threshold tuning, versioned model metadata, and tracking files are stored under <code>artifacts/</code> .	Shows real experiment evidence, not just final screenshots.
Implementation: deployment and monitoring	Docker Compose topology, health checks, Prometheus metrics, alert rules, Grafana dashboards, and MLflow runtime tracking are all implemented.	Strong end-to-end MLOps evidence for a course project.
Testing and CI/CD	50 passing tests, 80% coverage, CI workflows, and Docker validation workflow are documented with current evidence.	Lets the team defend engineering quality with concrete numbers instead of vague claims.
Responsible AI	SHAP artifacts, uncalibrated-score disclosure, human review workflow, and ethical limitations are explicitly documented.	Honest discussion is stronger than overstating fairness or explainability that the dataset cannot support.
Documentation	README, ARCHITECTURE, CONTRIBUTING, OpenAPI docs, and this consolidated report are present.	Reviewers can navigate the repo quickly and map claims back to code and artifacts.

Table 21: Grading-rubric alignment matrix for evaluators and presentation preparation.

29 Limitations

- Dataset limitation: the Kaggle dataset is a public benchmark and does not represent the full richness of real banking transactions.
- PCA feature opacity: `V1–V28` are not business-interpretable features.
- In-memory persistence by default: the primary demo path stores `alert` and `case` records in process memory.
- Security maturity gap: token-based role checks, rate limiting, and audit events exist, but enterprise-grade IAM and hardened RBAC governance are not fully implemented.
- No drift detection: the repository does not implement live data drift, concept drift, or automated retraining triggers.
- No closed feedback loop: resolved fraud outcomes are not yet wired into a model retraining pipeline.
- Runtime verification gap: not all live-stack deployment paths were re-run in this report update.

- Identity-mapping gap: the repository history currently exposes the git author names **Duong Binh An** and **Annwyn**, so the final submission should clearly map those identities to the official member roster if aliases differ.
- Container hardening gap: the Docker images are functional and reproducible, but further hardening such as non-root execution and multi-stage optimization would improve production readiness.

30 Future Work

- Promote the SQL repository path into the default operational mode with durable database-backed **alert**, **case**, and timeline storage.
- Use resolved **case** outcomes as supervised feedback for monitoring and retraining.
- Add scheduled retraining, champion-challenger evaluation, and promotion controls.
- Monitor feature drift, score drift, queue drift, and outcome drift.
- Incorporate interpretable transactional features such as merchant, beneficiary, device, geography, and authentication context.
- Extend current token-role checks into stronger RBAC, identity integration, and hardened audit retention.
- Add calibration or percentile-based score presentation to reduce misuse of raw score values.
- Expand CI and deployment verification for the full container stack.
- Add analyst productivity and queue-aging analytics.

31 Conclusion

This repository implements a coherent real-time fraud detection system for banking-style transaction decision support. Its core contribution is the integration of machine learning scoring with operational workflow: the system computes an uncalibrated **risk_score**, maps it to a **risk_tier** and **decision_recommendation**, creates an **alert** and **case** for suspicious transactions, supports analyst investigation, and exposes monitoring and deployment components around that workflow.

The project therefore contributes more than a fraud model. It demonstrates how fraud analytics, backend services, frontend operations, and MLOps tooling can be combined into an end-to-end fraud decision-support system.

32 Documentation Cleanup Summary

This report consolidates the project narrative into a single source of truth. The documentation cleanup achieved the following:

- removed duplicated project narratives that previously existed across multiple Markdown reports
- standardized terminology around **risk_score**, **risk_tier**, **decision_recommendation**, **alert**, and **case**
- reconciled inconsistent statements about the deployed model by aligning to artifact truth
- reconciled persistence and security status by distinguishing configurable implementation from production hardening

Implementation Status Matrix

Capability	Status	Notes
ML scoring API	Implemented	Artifact-backed deployed model and inference path
<code>risk_tier</code> and <code>decision_recommendation</code> policy	Implemented	Explicit threshold and policy mapping
<code>alert</code> and <code>case</code> creation	Implemented	Triggered for REVIEW and HIGH
Investigation timeline	Implemented	Timeline endpoint and event model present
Frontend analyst workflow	Implemented	Dashboard, queue, case actions, and timeline
Prometheus and Grafana monitoring	Implemented	Metrics, rules, and dashboard assets present
MLflow runtime tracking	Implemented	Runtime traffic tracker and deployment service present
SQL/Postgres persistence	Implemented (configurable)	Repository path, migrations, and Compose wiring are available
Reason-code explainability	Partially implemented	Heuristic and policy-derived rather than causal
Streaming behavior	Demo-level	Pull-based simulator, not event-bus-backed banking ingestion
In-memory case persistence	Demo-level	Process-bound storage intended for local demo
Runtime deployment verification	Unverified in this update	Deployment assets exist, but the full live stack was not re-run here
Enterprise RBAC/IAM and audit hardening	Future work	Current controls are token-based and limited
Drift detection and automated retraining	Future work	Not implemented
Fairness validation	Future work	Limited by dataset scope and missing protected-attribute fields
Real banking feature enrichment	Future work	Current data source is benchmark-style and anonymized

Table 22: Implementation status matrix.

A Presentation and Demo Readiness

The presentation rubric evaluates more than whether the repository exists. It evaluates whether the team can explain the system coherently, demonstrate the product smoothly, and answer technical questions with evidence. For that reason, this appendix turns the report into a practical presentation-defense document rather than only a static write-up.

Recommended live demo sequence

1. Start with the problem statement: fraud is rare, high-stakes, and operationally constrained by analyst capacity.
2. Open `/docs` and show that the API is typed, documented, and exposes more than a single prediction endpoint.
3. Score one transaction through `POST /predict` and explain the difference between `risk_score`, `risk_tier`, and `decision_recommendation`.
4. Show that suspicious outcomes create `alert` and `case` records that can be listed and resolved through workflow endpoints.
5. Open the frontend dashboard and demonstrate queue review, case detail, status update, and timeline visibility.
6. Show Prometheus or Grafana to prove that the system is observable, not only functional.
7. Close by pointing to the current validation evidence: 50 passing tests, 80% coverage, Docker Compose deployment, and versioned model artifacts.

Technical deep-dive narrative

- Explain the architecture as a pipeline: request validation, scoring, policy mapping, workflow creation, and monitoring.
- Emphasize why the model output is treated as ranking rather than calibrated probability; this is one of the most important technical honesty points in the project.
- Defend the promoted model choice using artifact evidence: logistic regression slightly outperformed the LightGBM challenger on validation PR-AUC and review-tier recall.
- Show that the design is operational, not only predictive: the core value is creating useful review workflows under analyst-capacity constraints.
- Highlight engineering maturity: typed FastAPI schemas, deterministic error handling, token auth, rate limiting, SQL repository option, CI coverage gate, and containerized deployment.

Likely Q&A prompts and defensible answers

Likely question	Defensible answer	Evidence to point at
Why did you choose logistic regression instead of Light-GBM?	Because the promoted model is selected by validation PR-AUC first and review-tier recall second under the same top-K policy. Logistic regression performed slightly better on the repository's selection criteria and is simpler to inspect and serve.	Model comparison section, benchmark figures, and <code>model_selection_summary.json</code> .
Why is the threshold so high?	Because the score is used as a ranking signal and the threshold is capacity-driven. The objective is to flag roughly the top 1% for review and the top 0.2% for the highest-risk tier, not to mimic a calibrated fraud probability cutoff.	Threshold metadata in <code>model_info.json</code> and the decision-logic section of this report.
How do you know the system is reliable?	The repository has unit, data, model, integration, SQL-persistence, and frontend/API smoke tests. The latest verification run completed with 50 passing tests and 80% source coverage, which matches the CI gate.	Testing section, <code>ci.yml</code> , and local verification output.
Is this fair or explainable?	The dataset does not support demographic fairness claims, so the report explicitly avoids overstating that. Explainability is provided through SHAP artifacts, feature importance, heuristic reason codes, and human review rather than claiming causal certainty.	Responsible AI section and SHAP figure.
Is this production ready?	It is production-oriented in architecture, but still a course-project system. The repo includes SQL persistence, monitoring, and containerization, but enterprise IAM, drift detection, and regulatory controls remain future work.	Limitations, future work, and deployment sections.

Table 23: Presentation-defense prompts aligned to common instructor questions.

B Submission Hardening Checklist

Before final submission and presentation, the team should confirm the following:

- the final PDF, README, ARCHITECTURE, and CONTRIBUTING documents all tell the same story
- the model artifact version referenced in the report matches the artifact currently committed in `artifacts/models/`
- each member can explain at least one owned component in code-level detail
- the demo environment is prepared with API, frontend, and monitoring URLs already known

- commit identities and official course-member names are mapped clearly if aliases differ